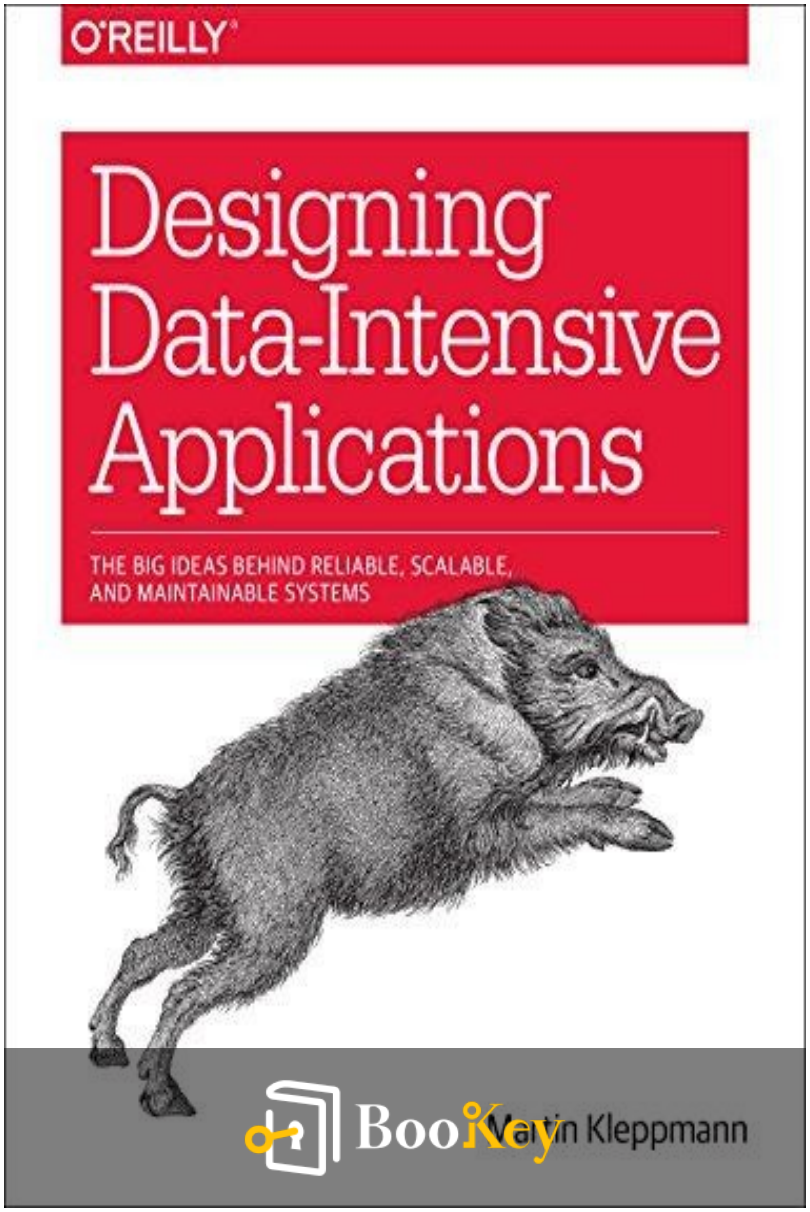
Designing Data-Intensive Applications PDF

Martin Kleppmann



More Book Summary
Scan to Download
Listen summary

Designing Data-Intensive Applications

Mastering the Choices for Effective Data-Intensive Application Design

Written by Bookey

Check more about Designing Data-Intensive Applications Summary

Listen Designing Data-Intensive Applications Audiobook

More Book Summary

Scan to Download

Listen summary

About the book

In today's technology landscape, data lies at the heart of critical system design challenges, encompassing issues such as scalability, consistency, reliability, efficiency, and maintainability. With a myriad of options—ranging from relational databases to NoSQL datastores and various processing frameworks—making informed choices can be daunting. In "Designing Data-Intensive Applications," Martin Kleppmann offers a practical and thorough examination of the strengths and weaknesses of different data technologies. This insightful guide empowers software engineers and architects to navigate the complexities of modern data systems, understand essential principles, and make informed decisions that optimize their applications. Readers will gain valuable insights into the fundamental trade-offs in consistency, scalability, and fault tolerance, while also exploring the foundational research in distributed systems that underpins today’s databases and architectures.

More Book Summary





Scan to Download

Listen summary

About the author

Martin Kleppmann is a prominent computer scientist and software engineer renowned for his expertise in distributed systems, data architectures, and scalable applications. With a strong academic background, including a PhD in Computer Science from the University of Cambridge, he has contributed significantly to the fields of databases and data-intensive applications. Kleppmann has worked with leading technology companies and has been involved in open-source projects, allowing him to merge theoretical insights with practical implementations. His acclaimed book, "Designing Data-Intensive Applications," provides a comprehensive guide to modern data-handling techniques, making complex concepts accessible to practitioners and researchers alike. Through his writing and speaking engagements, Kleppmann continues to influence and educate the next generation of developers in the intricacies of managing and leveraging data effectively in today's digital landscape.

More Book Summary


Scan to Download
Listen summary

Ad

Scan to Download

Try Bookey App to read 1000+ summary of world best books

Unlock 1000+ Titles, 80+ Topics

New titles added every week

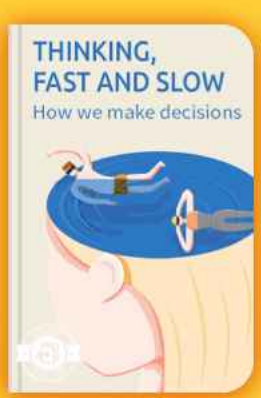
Brand | Leadership & Collaboration | Time Management | Relationship & Communication

ness Strategy | Creativity | Public | Money & Investing | Know Yourself | Positive P

Entrepreneurship | World History | Parent-Child Communication | Self-care | Mind & Spi

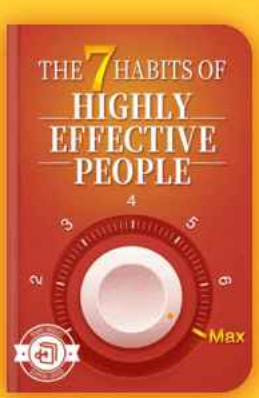
Insights of world best books















Free Trial with Bookey

Summary Content List

Chapter 1 : Reliable, Scalable and Maintainable Applications

Chapter 2 : Data Models and Query Languages

Chapter 3 : Storage and Retrieval

Chapter 4 : Encoding and Evolution

Chapter 5 : Part II. Distributed Data

Chapter 6 : Replication

Chapter 7 : Partitioning

Chapter 8 : Transactions

Chapter 9 : The Trouble with Distributed Systems

Chapter 10 : Consistency and Consensus

Chapter 11 : Part III. Derived Data

Chapter 12 : Batch Processing

Chapter 13 : Stream Processing

More Book Summary





Scan to Download

Listen summary

Chapter 1 Summary : Reliable, Scalable and Maintainable Applications



Section	Key Points
Introduction	- Applications are data-intensive rather than compute-intensive. - Challenges include the amount, complexity, and speed of data changes. - Applications use standard components like data storage, caching, and messaging.
Thinking About Data Systems	- Different tools serve unique purposes and performance characteristics. - Emerging tools blur lines between traditional categories of data systems. - Integration through application code is necessary for diverse data needs.
Key Objectives for Data Systems	- Reliability: Systems must function correctly despite faults. - Scalability: Systems need to effectively manage increased loads. - Maintainability: Systems should be easy for teams to work on over time.
Reliability	- Systems must perform expected functions and tolerate user mistakes. - Reliability involves designing systems to expect and manage faults.
Scalability	- Define current load to anticipate future increases. - Explore growth strategies: vertical scaling (upgrading) and horizontal scaling (adding machines).
Maintainability	- Focus on operability, simplicity, and evolvability. - Good software design enhances maintainability and supports future adaptations.

More Book Summary

Scan to Download

Listen summary

Section	Key Points
Conclusion	- Reliability, scalability, and maintainability are key for functional applications. - Design considerations and patterns are crucial for achieving these goals.

Chapter 1: Reliable, Scalable, and Maintainable Applications

Introduction

- Applications today are primarily data-intensive rather than compute-intensive.
- Data-related challenges often include the amount, complexity, and speed of data changes.
- Data-intensive applications are built using standard components for functionality such as data storage, caching, searching, messaging, and batch processing.

Thinking About Data Systems

- Different tools like databases and message queues serve unique purposes and performance characteristics.
- New tools are emerging that combine functionalities,

More Book Summary
Scan to Download
Listen summary

blurring the lines between traditional categories of data systems.
- Applications may require a mix of tools to meet diverse data handling needs, necessitating integration through application code.

Key Objectives for Data Systems

-
Reliability
: Systems should perform correctly despite faults. It encompasses fault tolerance and recognizing that all systems may experience faults (hardware failures, software bugs, human errors).
-
Scalability
: Systems must manage increased loads effectively. Scalability involves planning for growth and ensuring consistent performance as demands increase.
-
Maintainability
: Systems should be easy for various teams to work on over time, ensuring they remain operational and adaptable.

More Book Summary
Scan to Download
Listen summary

Reliability

- Essential aspects include performing expected functions, tolerating user mistakes, maintaining performance, and preventing unauthorized access.
- Reliability often involves designing systems that expect and manage faults.

Scalability

- Define current load parameters to understand how a system will handle future increases.
- Discuss growth strategies like vertical scaling (upgrading existing hardware) and horizontal scaling (adding more machines).

Maintainability

- Focus on operability (making routine tasks easy), simplicity (reducing complexity), and evolvability (ease of future changes).
- Good software design enhances maintainability, supporting future modification and adaptation.

More Book Summary
Scan to Download
Listen summary

Conclusion

- Reliability, scalability, and maintainability are crucial for functional applications, requiring intentional design considerations and patterns.
- Future sections of the book will delve into specific systems and techniques used to achieve these goals.

More Book Summary

Scan to Download
Listen summary

Example

Key Point:Understanding reliability is crucial when designing data systems.

Example:Imagine you are developing a financial application that must process thousands of transactions daily. If one transaction fails due to a minor glitch or unexpected user input, it could lead to incorrect account balances, customer dissatisfaction, and regulatory issues. By focusing on reliability, you implement mechanisms that not only catch such errors but also allow the application to recover automatically, ensuring customers can always rely on accurate information. This means incorporating features like error handling, transaction logging, and redundancy to keep the system operational even when unexpected faults occur.

More Book Summary
Scan to Download
Listen summary

Critical Thinking

Key Point:Reliability, scalability, and maintainability are critical in the design of data-intensive applications.

Critical Interpretation:While the author emphasizes these principles as central to creating successful data systems, it is essential to interrogate this perspective. Reliability is often prioritized, but the fluctuating nature of modern data demands may sometimes require trade-offs in other areas, such as performance or complexity. Moreover, the assertion that systems must always be scalable raises questions about the practical limitations and costs associated with scaling; not all applications may benefit from extensive scalability, and in some cases, a more robust architecture built on less fluid models could suffice. Furthermore, maintainability, while important, is often subjective and can vary significantly across teams and organizations, raising doubts about the applicability of the author's definitions. This critique reflects a broader discussion in software engineering literature, notably illustrated in works by authors like Simon Brown, who emphasizes architectural trade-offs and the importance of context in

More Book Summary

Scan to Download
Listen summary

software design.

Chapter 2 Summary : Data Models and Query Languages



Chapter 2: Data Models and Query Languages

The discussion begins with the assertion that data models are crucial in software development, greatly influencing not just coding but also our problem-solving perspective. This chapter explores the various layers of data representation necessary for applications, going from real-world modeling to the use of different data storage formats like JSON, XML, and relational databases.

Key Concepts in Data Models

More Book Summary

Scan to Download

Listen summary

1.

Layered Data Models

: Each layer of an application requires effective abstractions that simplify interactions with lower layers, allowing teams like application developers and database engineers to collaborate efficiently.

2.

Types of Data Models

:

-

Relational Model

: Revolutionized by Edgar Codd in 1970, this model organizes data into tables (relations), emphasizing ease of use and implementation. Over the decades, it has adapted well to various application needs, becoming dominant in data storage.

-

Document Model

: Emerged as a flexible alternative, particularly suited to self-contained documents. It addresses the need for less rigid data structures compared to the relational model, reducing the impedance mismatch common in object-oriented programming.

More Book Summary

Scan to Download

Listen summary

-

Graph-Based Models

: Ideal for applications with intricate relationships (many-to-many). The chapter details property graphs and introduces query languages such as Cypher and SPARQL for manipulating graph data.

Comparative Analysis: Relational vs. Document Models

-

Impedance Mismatch

: Describes the disparities between application objects and relational database representations, often resulting in complexities. Object-relational mapping helps, but some fundamental differences remain inherent.

-

Schema Flexibility

: Document databases usually adopt a schema-on-read approach, which is more adaptable than the schema-on-write of relational databases. This flexibility aids changing application requirements, albeit at the risk of inconsistent data.)

More Book Summary

Scan to Download

Listen summary

-

Data Locality

: Storing data in a document format may improve performance when fetching entire documents due to reduced queries, but operations affecting document size can be unwieldy.

Graph-Like Data Models

-

Characteristics of Graphs

: Focus on vertices (nodes) and edges (relationships), offering a powerful framework for complex associations. Social networks and web structures exemplify applications well-suited to graph representation.

-

Query Languages

: Various languages exist for graph manipulation, including Cypher for Neo4j and SPARQL for RDF, allowing for sophisticated queries that express complex relationships succinctly.

Evolution of Query Languages

More Book Summary
Scan to Download
Listen summary

-

Declarative vs. Imperative

: The chapter distinguishes declarative languages like SQL from imperative forms, emphasizing the operational efficiencies of declarative syntax and its suitability for database optimizations.

-

Emeriling Technologies

: As NoSQL databases gain traction, query languages continue to evolve, with features from successful models being integrated into newer systems—a convergence aimed at enhancing versatility and performance in database management.

Conclusion

Data models are pivotal in shaping application performance and usability. As practitioners navigate through relational, document, and graph paradigms, understanding their distinctions helps in choosing the appropriate model for specific requirements. The chapter also foreshadows upcoming discussions on the practical implementations and the common trade-offs involved with these models.

More Book Summary
Scan to Download
Listen summary

Chapter 3 Summary : Storage and Retrieval

Section	Key Points
Overview	Importance of understanding storage mechanisms for choosing suitable databases.
Key Concepts	Databases need to store and retrieve data; different engines serve OLTP and OLAP workloads.
Data Structures	Types: log-structured and page-oriented engines (e.g., B-trees); key-value stores demonstrate logs.
Indexes	Indexes improve data retrieval efficiency; trade-offs between read/write performance.
Log-Structured and Page-Oriented Storage	Log-structured systems enhance write performance by sequentializing writes; page-oriented focuses on fixed-size blocks.
LSM-Trees and SSTables	LSM-Trees are efficient for high write volumes; SSTables store sorted data for efficient reads/writes.
B-trees	Efficient in relational databases for indexed data management.
Multi-column and Fuzzy Indexes	Support queries across multiple fields and searching for similar keys.
In-Memory Databases	Enhance performance with durability via logs/snapshots; use of anti-caching for larger datasets.
Transaction Processing vs. Analytics	OLTP for high-frequency transactions; OLAP for analyzing large datasets separately.
Data Warehousing	ETL processes vital for structuring historical data; specific schemas for efficient querying.
Column-oriented Storage	Effective for large datasets, optimizing retrieval with compression and bitmap indexing.
Conclusion	Choosing the correct storage approaches is crucial for optimizing workload-specific database performance.

Storage and Retrieval

Overview

More Book Summary





Scan to Download

Listen summary

This chapter discusses the foundational aspects of how databases store and retrieve data. It emphasizes the importance for application developers to understand the underlying mechanisms of storage engines to choose the best one for their applications.

Key Concepts

- A database fundamentally needs to store data and retrieve it upon request.
- Different storage engines are optimized for different workloads, particularly transactional (OLTP) and analytical (OLAP) workloads.

Data Structures

- Traditional databases often utilize two families of storage engines: log-structured storage engines and page-oriented

Install Bookey App to Unlock Full Text and Audio

More Book Summary

Scan to Download

Listen summary

Ad

Scan to Download



Why Bookey is must have App for Book Lovers

30min Content
The deeper and clearer interpretation we provide, the better grasp of each title you have.

Text and Audio format
Absorb knowledge even in fragmented time.

Quiz
Check whether you have mastered what you just learned.

And more
Multiple Voices & fonts, Mind Map, Quotes, IdeaClips...

Free Trial with Bookey

Chapter 4 Summary : Encoding and Evolution

CHAPTER 4: Encoding and Evolution

Introduction

Everything changes over time, including applications. Building systems that are easy to adapt to changes, referred to as evolvability, is critical. Changes to application features often require updates to the stored data, necessitating shifts in data models that can handle such evolution.

Data Models and Schema Evolution

-
Relational Databases
: Operate under a single schema which can be altered, but only one schema is enforced at any given time.
-
Schema-on-Read Databases

More Book Summary
Scan to Download
Listen summary

: Do not enforce schemas, allowing for more flexibility in data representation.

Compatibility Requirements

When changing data formats or schemas, the application code must often be updated. However, this cannot always happen instantaneously, leading to the need for backward and forward compatibility:

-
Backward Compatibility
: Newer code can read data produced by older code.

-
Forward Compatibility
: Older code can read data produced by newer code.

Data Encoding Formats

Several data encoding formats are examined, which include JSON, XML, Protocol Buffers, Thrift, and Avro. Each has different mechanisms for handling schema changes and supporting coexistence of old and new data.

Encoding Mechanisms

More Book Summary
Scan to Download
Listen summary

Data representations in memory must be encoded into byte sequences for storage or network transmission. This process is known as encoding (serialization), while the reverse is decoding (deserialization).

Language-Specific Formats

While many programming languages offer built-in serialization capabilities, they often tie the encoded data to a specific language, creating challenges for data interchange across systems.

Standardized Encodings

-
JSON and XML
: Common encoding formats but can be ambiguous with data types, particularly around numbers and binary strings. They optionally support schemas, though compliance can vary.
-
CSV
: A simple format but lacks schema support, making the interpretation of data more manual and error-prone.

More Book Summary
Scan to Download
Listen summary

Binary Encoding

For internal use, binary formats outperform standardized textual formats in terms of efficiency, thus several binary encodings for JSON and XML have been introduced. However, their adoption has not matched that of textual formats.

Protocol Buffers and Thrift

Both of these frameworks rely on schemas for data encoding. They allow for code generation from schemas, ensuring efficient and structured data handling in various programming languages.

Avro

Originating from Hadoop, Avro encodes data without tag numbers in its schema, and supports schema evolution by allowing a writer's schema and reader's schema to differ as long as they are compatible.

Maintaining Schema Evolution

More Book Summary
Scan to Download
Listen summary

Understanding schema changes, field additions/removals, and datatype adjustments is essential. Backward and forward compatibility needs are satisfied through careful schema design, including the use of union types and default values.

Data Flow Modes

Different scenarios in which data exchange occurs:

-

Databases

: Involves writing and reading data through encoding/decoding processes, requiring compatibility during updates.

-

RPC and REST APIs

: Communication occurs over various data encodings, necessitating careful management of request and response compatibility.

-

Asynchronous Messaging

: Utilizes message brokers that temporarily store messages, allowing for decoupled, one-way communication.

More Book Summary





Scan to Download

Listen summary

Conclusion

It is crucial to encode data effectively to facilitate rolling upgrades and schema evolution, enabling changes without significant disruptions. By maintaining compatibility across different processes through careful encoding formats, data flow remains efficient and adaptable to evolving application needs.

More Book Summary
Scan to Download
Listen summary

Critical Thinking

Key Point:Data encoding and schema evolution are fundamental for the longevity of software applications.

Critical Interpretation:While Kleppmann emphasizes the importance of designing applications with robust schema evolution capabilities, it is worth questioning whether the reliance on schema flexibility might lead to unintended complexity in data management. The assumption that developers can manage evolving schemas without substantial overhead could be overly optimistic. This perspective prompts a critical examination of other frameworks or methodologies that offer simpler solutions for evolution without necessarily overhauling data models, such as functional programming techniques that prioritize immutability (see 'Functional Programming in Scala' by Paul Chiusano and Rúnar Bjarnason). Evaluating the trade-offs between adaptiveness and maintainability could provide valuable insights into alternative methods that might challenge the author's conclusions.

More Book Summary





Scan to Download

Listen summary

Chapter 5 Summary : Part II. Distributed Data

Section	Summary
Introduction to Distributed Systems	Explores multi-machine data systems, emphasizing scalability, fault tolerance, high availability, and reduced latency for global users.
Scaling Approaches	Vertical Scaling: Upgrading to powerful machines; limited scalability and super-linear costs. Shared-Disk Architecture: Multiple machines share disks but have independent resources; limited by contention. Shared-Nothing Architecture: Independent machines with their own resources; supports geographic distribution and datacenter survival, allowing for various hardware configurations.
Key Considerations in Shared-Nothing Architectures	Increased complexity for developers, limitations in data models, and some simple architectures can outperform complex systems.
Data Distribution Mechanisms	Replication: Copies of data across nodes for redundancy and performance, discussed in Chapter 5. Partitioning: Splitting databases into partitions for distribution (sharding), further explored in Chapter 6.
Conclusion	Prepares readers for trade-offs in distributed systems, discusses transactions in Chapter 7, addresses fundamental limitations in Chapters 8 and 9, and covers integration of datastores in Part III.

Distributed Data

Introduction to Distributed Systems

In Part II of the book, we explore data systems that utilize

More Book Summary
Scan to Download
Listen summary

multiple machines for storage and retrieval, following the discussion of single-machine systems in Part I. The key reasons for distributing databases include scalability, fault tolerance, high availability, and reduced latency for global users.

Scaling Approaches

1.
Vertical Scaling

- Involves upgrading to more powerful machines (vertical scaling).
- Single machines can comprise many CPUs, RAM, and disks under one operating system.
- However, costs can be super-linear, and scalability has limits.

2.
Shared-Disk Architecture

- Multiple machines connect to shared disks but have independent CPUs and RAM.
- This method is suitable for some data warehousing but has scalability limitations due to contention.

More Book Summary
Scan to Download
Listen summary

3.

Shared-Nothing Architecture

- Popular architecture where each machine operates independently with its own resources.
- Allows for potential distribution across geographic locations and survival of datacenter failures.
- Offers flexibility in using various price/performance machines.

Key Considerations in Shared-Nothing Architectures

- This architecture incurs additional complexity for application developers and may limit expressive data models.

- In some cases, simpler architectures can outperform complex distributed systems.

Data Distribution Mechanisms

1.

Replication

More Book Summary
Scan to Download
Listen summary

- Involves maintaining copies of the same data across several nodes for redundancy and performance improvements.
- Discussed in detail in Chapter 5.

2.

Partitioning

- Solidifies the concept of splitting databases into smaller subsets (partitions) for distribution across nodes (sharding).
- Further explored in Chapter 6.

Conclusion

- This part prepares readers for understanding the trade-offs in distributed systems and discussions on transactions in Chapter 7.
- Fundamental limitations of distributed systems will be addressed in Chapters 8 and 9, while Part III will cover integrating multiple datastores into a comprehensive system.

More Book Summary
Scan to Download
Listen summary

Example

Key Point:Understanding the benefits and challenges of distributed systems is crucial for effective data management.

Example:Imagine you are designing a global e-commerce platform where customers from varying regions expect quick access to product information. To accommodate varying workloads, you decide to use a shared-nothing architecture. Each continent hosts its own independent cluster of servers, reducing latency and enhancing fault tolerance. However, as you scale this setup, you grapple with increased complexity in application development and data consistency across nodes. This duality captures the essence of distributed systems; they offer extensive scalability and resilience, yet demand profound consideration of their inherent challenges.

More Book Summary



Scan to Download

Listen summary

Chapter 6 Summary : Replication

Replication

Replication involves maintaining copies of the same data across multiple machines connected via a network. There are several motivations for data replication:
- To keep data geographically close to users, reducing latency.
- To increase availability by allowing the system to continue functioning even if some parts fail.
- To improve read throughput by scaling out the number of machines serving read queries.
This chapter starts by discussing simple replication where each node holds a full copy of the dataset, and will later address partitioning large datasets across multiple nodes. Key topics covered include synchronizing data changes, different replication algorithms, and handling various faults.

Types of Replication Algorithms

1.
Single-Leader Replication

More Book Summary
Scan to Download
Listen summary

: One replica acts as the leader (master) while others (followers) replicate writes from the leader. Clients send write requests to the leader, which updates its local storage and informs followers.

2.

Multi-Leader Replication

: Multiple nodes can accept writes, which are then replicated among them. This configuration allows for greater availability and resilience against failures, especially in multi-datacenter environments, but introduces the need for conflict resolution.

3.

Leaderless Replication

: Any replica can accept writes from clients. The system must handle concurrent updates and ensure that clients can read from multiple replicas to avoid stale data issues.

Synchronous vs. Asynchronous Replication

Install Bookey App to Unlock Full Text and Audio

More Book Summary

Scan to Download

Listen summary

Ad

Scan to Download

App Store
Editors' Choice

22k 5 star review

Positive feedback

Sara Scholz

tes after each book summary
erstanding but also make the
and engaging. Bookey has
ding for me.

Fantastic!!!

Masood El Toure

I'm amazed by the variety of books and languages Bookey supports. It's not just an app, it's a gateway to global knowledge. Plus, earning points for charity is a big plus!

Fi

Ab
bo
to
m

José Botín

ding habit
p's design
ual growth

Love it!

Wonnie Tappkx

Bookey offers me time to go through the important parts of a book. It also gives me enough idea whether or not I should purchase the whole book version or not! It is easy to use!

Time saver!

Bookey is my go-to app for
summaries are concise, ins
curated. It's like having acc
right at my fingertips!

Awesome app!

Rahul Malviya

I love audiobooks but don't always have time to listen to the entire book! bookey allows me to get a summary of the highlights of the book I'm interested in!!! What a great concept !!!highly recommended!

Beautiful App

Alex Walk

This app is a lifesaver for book lovers with busy schedules. The summaries are spot on, and the mind maps help reinforce wh I've learned. Highly recommend!

Free Trial with Bookey

Chapter 7 Summary : Partitioning

Partitioning Overview

Partitioning is a method used to divide a large database into smaller, more manageable pieces, allowing for improved scalability and distribution of data across nodes in a system. It differs from network partitioning, which deals with faults in networks.

Key Concepts in Partitioning

1.
Objectives of Partitioning
: The primary goal of partitioning is scalability, enhancing performance by distributing data and queries across multiple nodes. This allows systems to handle larger datasets and higher query throughput by utilizing shared-nothing architecture.
2.
Terminology Confusion
: Different databases use various terms for partitioning, such as shards in MongoDB and regions in HBase. Despite the

More Book Summary
Scan to Download
Listen summary

terminology, the concept largely remains consistent across systems.

3.

Combination with Replication

: Partitioning is often used in conjunction with replication, where copies of each partition are stored on multiple nodes to ensure fault tolerance.

Approaches to Partitioning

1.

Key Range Partitioning

: This approach involves assigning continuous ranges of keys to different partitions, which allows efficient range queries but has risks of hot spots.

2.

Hash Partitioning

: Utilizes a hash function to determine which partition holds a particular key, helping to evenly distribute data but making range queries less efficient.

3.

Document vs. Term-based Partitioning

: In the context of secondary indexes, data can be indexed per document or by term, impacting query performance and

More Book Summary
Scan to Download
Listen summary

complexity.

Rebalancing Partitions

Over time, as nodes are added or removed or data volumes change, rebalancing partitions ensures data load remains evenly distributed across nodes. Strategies include:

-

Fixed Number of Partitions

: Initially creating more partitions than nodes allows flexibility when nodes are added or removed.

-

Dynamic Partitioning

: Automatically splits or merges partitions based on load and usage, adapting to changes in data volume.

Request Routing

When queries are made, it’s crucial to route requests to the correct partitions. This may be achieved through:

1. Client-aware routing, where clients directly connect to nodes.
2. A dedicated routing layer that forwards requests based on current partition assignments.

More Book Summary





Scan to Download

Listen summary

Conclusions

The chapter highlights the importance of partitioning in managing large datasets effectively. Choosing an appropriate partitioning scheme and robust rebalancing methods is essential for maintaining system performance and reliability. Additionally, the integration with secondary indexes and request routing contributes to overall efficiency.
Future discussions in the book will address transaction handling and the complexities involved in multi-partition write operations.

More Book Summary

Scan to Download
Listen summary

Chapter 8 Summary : Transactions

Chapter 7: Transactions

Overview of Transactions

- Transactions are essential for managing data consistency, concurrency, and error handling in applications interacting with databases. They allow various operations to be grouped and executed as a single unit, ensuring all-or-nothing outcomes (commit or abort) to maintain data integrity.

Common Issues in Data Management

- Data systems face several challenges, including hardware or software failures, network interruptions, concurrent writes from multiple clients, and race conditions leading to inconsistent states. Implementing fault tolerance is critical yet complex.

Safety Guarantees of Transactions (ACID)

More Book Summary
Scan to Download
Listen summary

- Transactions are characterized by four main properties:
  -
Atomicity
: All operations in a transaction succeed or none do.
  -
Consistency
: Transactions bring the database from one valid state to another.
  -
Isolation
: Concurrent transactions do not affect each other's execution.
  -
Durability
: Once a transaction commits, it persists, despite system failures.

Isolation Levels

- Isolation levels categorize how transaction operations are handled in the presence of concurrent transactions. Common levels include:
  -
Read Committed
: Prevents dirty reads.

More Book Summary
Scan to Download
Listen summary

-

Snapshot Isolation

: Allows reads from a consistent snapshot of the database, preventing many concurrency issues but potentially allowing contradictory states (e.g., write skew).

-

Serializable Isolation

: The strongest level that prevents all concurrency issues, but often less efficient.

Concurrency Control Techniques

- Two main types:

-

Pessimistic Concurrency Control

: Locks resources to prevent conflicts (e.g., Two-Phase Locking).

-

Optimistic Concurrency Control

: Assumes conflicts are rare, allowing transactions to proceed without locking but checking for conflicts before committing (e.g., Serializable Snapshot Isolation).

Handling Errors and Aborts

More Book Summary
Scan to Download
Listen summary

- Robust error handling mechanisms enable safe retries of transactions. Notable considerations include distinguishing between transient and permanent errors, handling side-effects, and ensuring data integrity across retries.

Weak Isolation Levels and Their Risks

- Weak isolation levels allow some anomalies to persist. Applications may need to implement additional safeguards to manage issues like lost updates, dirty reads, and write skew.

Serializable Isolation Techniques

- The chapter concludes with a discussion of approaches for ensuring serializable transactions, such as serial execution, two-phase locking (2PL), and Serializable Snapshot Isolation (SSI). Each method has distinct performance implications and use cases, affecting system throughput and latency.

Final Thoughts

- Transactions play a pivotal role in ensuring data integrity amid concurrent access. While NoSQL databases may

More Book Summary
Scan to Download
Listen summary

sacrifice transactional guarantees for scalability, applications built on consistent transactional models benefit from reduced complexity in data management.

References

- Includes citational references for further reading and research on applicable algorithms and concepts within database transaction management and concurrency control.

More Book Summary
Scan to Download
Listen summary

Example

Key Point:Understanding the Importance of Atomicity in Transactions

Example:Imagine you're processing an online order: if you attempt to deduct funds from a customer's account but fail to reserve the product, atomicity ensures the entire order fails rather than leaving inconsistencies in your system.

More Book Summary

Scan to Download

Listen summary

Critical Thinking

Key Point:Importance of Transactions in Data Management

Critical Interpretation:While Kleppmann emphasizes the crucial role of transactions in ensuring data integrity during concurrent operations, it is important to question whether a strict adherence to ACID properties is necessary for all applications. For instance, systems designed for high availability and scalability, like NoSQL databases, may sacrifice certain transaction guarantees, arguing that eventual consistency can be sufficient in many scenarios (Brewer's CAP Theorem). This perspective challenges the need for complex transaction management in less critical systems. Furthermore, research such as 'Database Scale and Consistency Trade-offs' suggests alternative approaches to managing data that may not align with traditional transaction models.

More Book Summary





Scan to Download

Listen summary

Chapter 9 Summary : The Trouble with Distributed Systems

Chapter 9: The Trouble with Distributed Systems

Overview of Faults in Distributed Systems

- The chapter begins with the assumption that faults are non-Byzantine, indicating realistic expectations about system failures.
- Throughout previous chapters, fault handling has been optimistic. The chapter acknowledges that anything that can go wrong will likely go wrong.

Understanding Challenges in Distributed Systems

- Distributed systems differ from single computers in that they face numerous potential issues due to network and hardware complexities.
- Engineers must design systems that fulfill user expectations even when components fail.

More Book Summary
Scan to Download
Listen summary

Faults and Partial Failures

- On a single computer, software is deterministic; it either works or it doesn’t. In contrast, distributed systems can experience partial failures where some parts are operational while others may fail unpredictably.
- This non-determinism is a key difficulty in managing distributed systems.

Network Issues

- Distributed systems utilize asynchronous packet networks, leading to various network-related failures including lost or delayed messages.
- The chapter emphasizes the importance of understanding that network reliability can’t be taken for granted and strategies must be in place for handling uncertainties.

Install Bookey App to Unlock Full Text and Audio

More Book Summary





Scan to Download

Listen summary

Ad

Scan to Download



Read, Share, Empower

Finish Your Reading Challenge, Donate Books to African Children.

The Concept

BOOKS FOR AFRICA × ×

This book donation activity is rolling out together with Books For Africa. We release this project because we share the same belief as BFA: For many children in Africa, the gift of books truly is a gift of hope.

The Rule

Earn 100 points → **Redeem a book** → **Donate to Africa**

Your learning not only brings knowledge but also allows you to earn points for charitable causes! For every 100 points you earn, a book will be donated to Africa.

Free Trial with Bookey

Chapter 10 Summary : Consistency and Consensus

Chapter 10: Consistency and Consensus

Introduction

In distributed systems, faults are inevitable, and handling them effectively is crucial for maintaining service availability and correctness. This chapter focuses on strategies for building fault-tolerant distributed systems, particularly through consensus algorithms.

Fault-Tolerant Systems

To ensure a distributed system continues to function correctly despite component failures, it is important to use general-purpose abstractions that provide useful guarantees. Consensus is one of the essential abstractions, where all nodes agree on a value despite failures and network issues, preventing issues like “split brain” scenarios.

More Book Summary
Scan to Download
Listen summary

Consistency Guarantees

In replicated databases, inconsistencies can occur due to delays in propagation of write operations. Most systems offer eventual consistency, meaning replicas will eventually converge to the same state. This section will dissect stronger consistency models that ensure a more coherent view of data compared to weaker models like eventual consistency.

Linearizability

Linearizability is a strong consistency model that appears to give the illusion of a single copy of data. Operations in a linearizable system appear to occur instantaneously at some point in time, ensuring that all clients see the same value for a given operation. The chapter delves into examples of non-linearizable systems and illustrates the implications of linearizability, emphasizing how it simplifies reasoning about distributed systems.

Consensus Algorithms

The consensus problem is critical to resolving conflicts

More Book Summary
Scan to Download
Listen summary

between nodes and ensuring that all nodes can agree on the same value. Various consensus algorithms, like Paxos and Raft, are introduced. These algorithms require the system to maintain a unique leader and facilitate orderly decision-making among nodes.

Distributed Transactions and Atomic Commitment

This section discusses the atomic commitment problem, detailing the two-phase commit (2PC) protocol which ensures that all participants in a distributed transaction either commit their changes or abort them entirely. The challenges of coordinator failures and how 2PC handles these scenarios are addressed, cautioning against its limitations and delay considerations.

Limitations and Improvements to Consensus

Consensus algorithms often come at a performance cost, leading to discussions surrounding the CAP theorem, which elucidates the trade-offs between consistency, availability, and partition tolerance. Additionally, account is taken of the issues that arise from network partitioning and delays, reinforcing the importance of having a robust consensus

More Book Summary
Scan to Download
Listen summary

implementation.

Membership and Coordination Services

Frameworks like ZooKeeper and etcd serve as tools for achieving consensus and coordinating distributed systems. These systems provide not just consensus but also facilitate configuration management, fault recovery, and synchronization across distributed nodes.

Summary

The chapter comprehensively covers significant concepts of consistency and consensus in distributed systems. It elucidates how these concepts are necessary for building resilient and reliable applications while also addressing the inherent trade-offs and limitations faced in practice. The importance of understanding these theories is highlighted, as they inform practical implementations in distributed architectures.

References

An extensive list of academic and technical references

More Book Summary
Scan to Download
Listen summary

supports the discussions in the chapter, providing resources for further exploration into these complex topics.

More Book Summary
Scan to Download
Listen summary

Chapter 11 Summary : Part III. Derived Data

Part III: Derived Data

Introduction

In Parts I and II of this book, the fundamental elements of a distributed database were discussed, covering various topics from data layout to distributed consistency under fault conditions. However, real-world applications frequently involve multiple databases to meet diverse data access and processing needs. This part will delve into the complexities of integrating various data systems, which often have different data models and access optimizations, into a unified architecture.

Systems of Record and Derived Data

Data storage and processing systems can be classified into two main categories:

More Book Summary
Scan to Download
Listen summary

1.

Systems of Record

- Also referred to as the source of truth, these systems maintain the authoritative version of data. New data is initially written here, ensuring that each fact is represented uniquely (typically normalized). Discrepancies with other systems are resolved in favor of the system of record.

2.

Derived Data Systems

- These systems produce data by transforming or processing existing data from other systems. Derived data, which can be recreated from the original source, includes caches, denormalized values, indexes, and materialized views. While derived data may seem redundant, it is crucial for enhancing the performance of read queries and allows for different perspectives on the same data.

Importance of Distinctions

Not all systems maintain a clear separation between systems of record and derived data, yet making this distinction clarifies data flow within the system. Understanding which

More Book Summary
Scan to Download
Listen summary

components produce and rely on specific data inputs and outputs is essential for building coherent and efficient architectures.

Role of Databases

Most databases and query languages themselves are not inherently classified as either systems of record or derived data systems. Rather, their classification depends on how they are utilized in a given application. A clear understanding of the data derivation relationships can help simplify complex system architectures.

Overview of Chapters

Chapter 10 discusses batch-oriented dataflow systems like MapReduce, illustrating their utility in building large-scale data architectures. Chapter 11 applies these principles to data streams, which provide similar capabilities with reduced latency. The final chapter explores concepts for constructing reliable, scalable, and maintainable applications in the future.

More Book Summary
Scan to Download
Listen summary

Chapter 12 Summary : Batch Processing

Summary of Chapter 10: Batch Processing

Introduction to Batch Processing

Batch processing involves executing jobs that handle large amounts of data without requiring user interaction. Unlike online systems that respond to immediate requests, batch systems are designed to process data over extended periods, typically without dedicated users waiting for results.

Types of Systems

1.
Online Systems
: They respond quickly to user requests, often measured by response times.
2.
Batch Processing Systems
: These systems process large volumes of data at once and are usually scheduled to run at specified intervals (e.g.,

More Book Summary
Scan to Download
Listen summary

nightly).
3.
Stream Processing Systems
: These operate on data streams in near real-time, able to react to incoming data as it arrives.

MapReduce and Its Historical Context

MapReduce is a batch processing model that has influenced many distributed data processing frameworks. It facilitates scalability and efficiency in processing large data sets, although its straightforward design can sometimes oversimplify complex tasks.

Batch Processing with Unix Tools

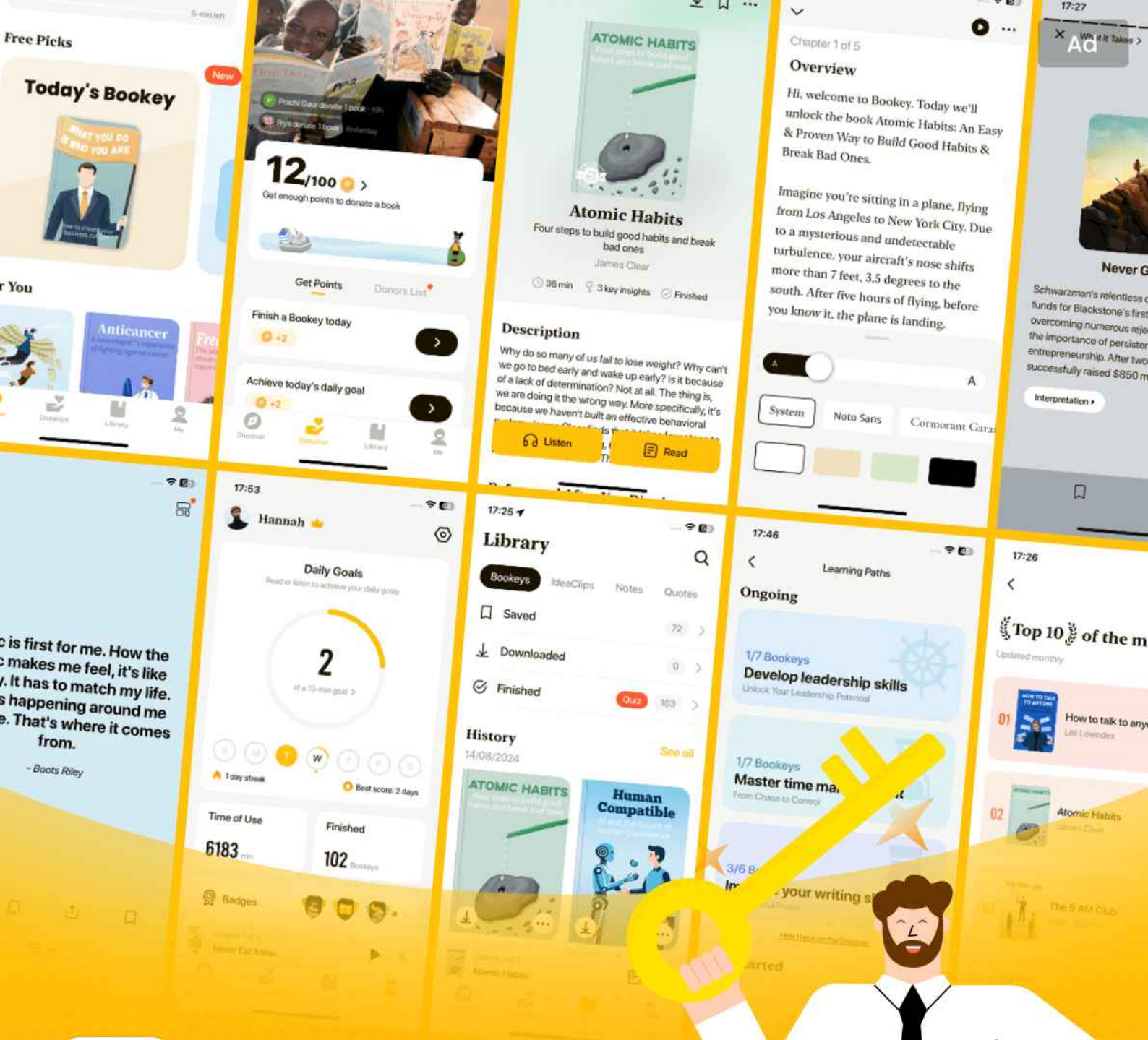
Unix tools embody a design philosophy of doing one task well and connecting processes through standardized

Install Bookey App to Unlock Full Text and Audio

More Book Summary

Scan to Download

Listen summary



World' best ideas unlock your potential

Free Trial with Bookey

Download on the App Store

GET IT ON Google Play

Scan to download

Chapter 13 Summary : Stream Processing

Chapter 11: Stream Processing

Introduction to Stream Processing

- Stream processing is a response to the limitations of batch processing, which relies on finite input data that can be processed all at once. In contrast, stream processing deals with continuous, unbounded data.
- Events are the key units of processing in streaming, representing small, immutable records of occurrences.

Event Streams

- Streams can be represented as events containing timestamps, originating from various sources like user actions or sensor readings.
- The relationship between producers and consumers in streams is more dynamic than in batch processing, with

More Book Summary
Scan to Download
Listen summary

messaging systems serving to notify consumers of new events.

Transmitting Event Streams

- Messaging systems facilitate the delivery of events to consumers by breaking the process into producers (sending events) and consumers (receiving them).
- The design of messaging systems varies, with strategies to handle cases where producers send messages faster than consumers can process them.

Message Brokers

- A messaging broker helps manage message flow and persistence, enabling producers and consumers to operate independently and reliably.
- Message brokers can maintain data for longer, potentially allowing for historical queries.

Stream Processing Frameworks

- Streaming frameworks operate continuously, process data in real time, and must handle the infinite nature of data

More Book Summary
Scan to Download
Listen summary

differently from batch processing frameworks.
- Techniques like microbatching and checkpointing help manage failures and guarantee fault tolerance.

Fault Tolerance and Exactly-once Processing

- Achieving fault tolerance in stream processing is critical, where the output must accurately reflect the processing state even after failures.
- Various mechanisms exist to ensure exactly-once semantics, such as using idempotent operations, which ensure that repeated execution has no additional effect.

Processing Streams

- Stream processing operations, similar to batch processing, can include transformations, filtering, and aggregation.
- Important challenges in stream processing include dealing with time, managing state, and ensuring correct event ordering to prevent inconsistencies.

Types of Joins

- Stream processing supports different types of joins,

More Book Summary
Scan to Download
Listen summary

including stream-stream joins, stream-table joins, and table-table joins, adapted to handle the continuous nature of data.

Conclusion

- Stream processing is vital for applications that require real-time data handling, transforming traditional batch operations into a continuous model where streams can be processed incrementally.
- Both message brokers and databases have adapted to support stream processing, emphasizing the importance of keeping data consistent across different systems.

More Book Summary
Scan to Download
Listen summary

Ad

Scan to Download

Try Bookey App to read 1000+ summary of world best books

Unlock 1000+ Titles, 80+ Topics

New titles added every week

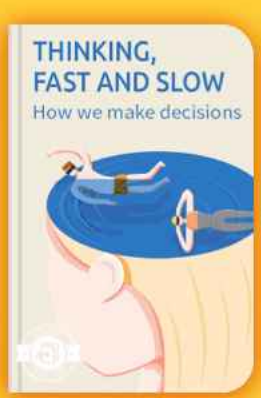
Brand | Leadership & Collaboration | Time Management | Relationship & Communication

ness Strategy | Creativity | Public | Money & Investing | Know Yourself | Positive P

Entrepreneurship | World History | Parent-Child Communication | Self-care | Mind & Spi

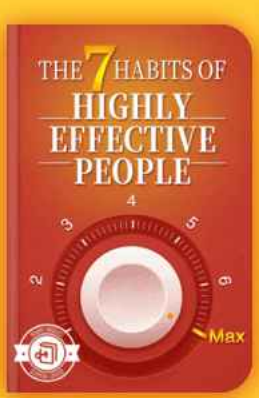
Insights of world best books















Free Trial with Bookey

Best Quotes from Designing Data-Intensive Applications by Martin Kleppmann with Page Numbers

View on Bookey Website and Generate Beautiful Quote Images

Chapter 1 | Quotes From Pages 23-46

1. When we combine several tools in order to provide a service, the service's interface or API usually hides those implementation details from clients.
2. Reliability means making systems work correctly, even when faults occur.
3. Scalability means having strategies for keeping performance good, even when load increases.
4. The major cost of software is not in its initial development, but in its ongoing maintenance.
5. Good operations can often work around the limitations of bad (or incomplete) software, but good software cannot run reliably with bad operations.
6. A single server system requires planned downtime if you need to reboot the machine... whereas a system that can

More Book Summary



Scan to Download

Listen summary

tolerate machine failure can be patched one node at a time, without downtime of the entire system.

Chapter 2 | Quotes From Pages 47-88

1. Data models are perhaps the most important part of developing software, because they have such a profound effect: not only on how the software is written, but also how we think about the problem that we are solving.
2. Building software is hard enough, even when working with just one data model, and without worrying about its inner workings. But since the data model has such a profound effect on what the software above it can and can’t do, it’s important to choose one that is appropriate to the application.
3. The relational model thus made it much easier to add new features to applications.
4. One model can be emulated in terms of another model, but the result is often awkward. That’s why we have different systems for different purposes, not a single one-size-fits-all

More Book Summary
Scan to Download
Listen summary

solution.

5. It seems likely that in the foreseeable future, relational databases will continue to be used alongside a broad variety of non-relational data stores,— an idea that is sometimes called polyglot persistence.
6. The lack of a schema is often cited as an advantage; we will discuss this in 'Schema flexibility in the document model'.
7. Graphs are good for evolvability: as you add features to your application, a graph can easily be extended to accommodate changes in your application’s data structures.
8. Declarative languages often lend themselves to parallel execution. Today, CPUs are getting faster by adding more cores, not by running at significantly higher clock speeds than before.

Chapter 3 | Quotes From Pages 89-128

1. On the most fundamental level, a database needs to do two things: when you give it some data, it should store the data—and when you ask it again

More Book Summary
Scan to Download
Listen summary

later, it should give the data back to you.

2. You need to select a storage engine that is appropriate for your application, from the many that are available.
3. If you want to efficiently find the value for a particular key in the database, we need a different data structure: an index.
4. Well-chosen indexes speed up read queries, but every index slows down writes.
5. You can then choose the indexes that give your application the greatest benefit, without introducing more overhead than necessary.
6. If you want to delete a key and its associated value, you have to append a special deletion record to the data file (sometimes called a tombstone).
7. In practice, you can maintain a hash map on disk, but... it is difficult to make an on-disk hash map perform well. It requires a lot of random access I/O, it is expensive to grow when it becomes full, and hash collisions require fiddly logic.

More Book Summary
Scan to Download
Listen summary

8. It’s easier to use a binary format which first encodes the length of a string in bytes, followed by the raw string (without need for escaping).
9. LSM-trees typically require a lot of disk bandwidth for compaction, and poor performance can occur when a query needs to read from multiple segments.
10. Since data is stored in sorted order, you can efficiently perform range queries... and because the disk writes are sequential, the LSM-tree can support remarkably high write throughput.

More Book Summary
Scan to Download
Listen summary

Download Bookey App to enjoy

1 Million+ Quotes

1000+ Book Summaries

Free Trial Available!

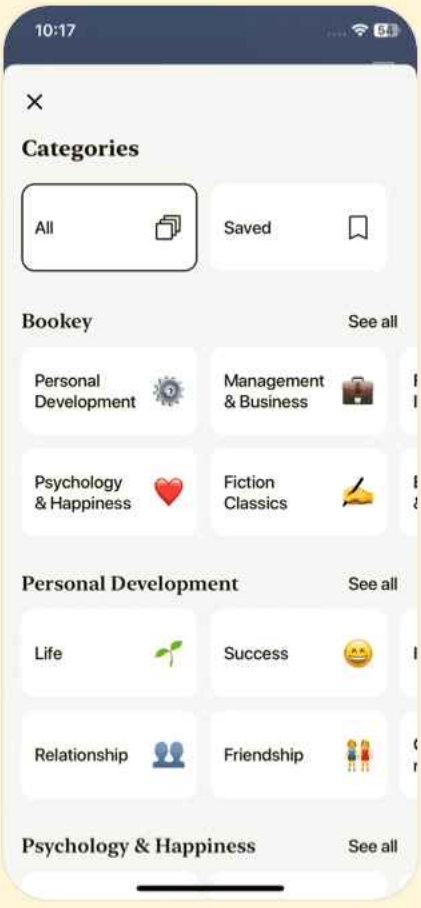
Scan to Download

Download on the App Store

GET IT ON Google Play







Chapter 4 | Quotes From Pages 129-162

1. Everything changes, and nothing stands still.”
   —Heraclitus of Ephesus, as quoted by Plato in Cratylus (360 BC)
2. Backward compatibility and forward compatibility are what make a system evolvable.
3. Data outlives code.
4. Schema evolution allows the same kind of flexibility as schemaless/schema-on-read JSON databases provide, while also providing better guarantees about your data and better tooling.
5. May your application's evolution be rapid and your deployments be frequent.

Chapter 5 | Quotes From Pages 163-166

1. For a successful technology, reality must take precedence over public relations, for nature cannot be fooled." —Richard Feynman
2. If your data is distributed across multiple nodes, you need to be aware of the constraints and trade-offs that occur in

More Book Summary
Scan to Download
Listen summary

such a distributed system— the database cannot magically hide these from you.

3. While a distributed shared-nothing architecture has many advantages, it usually also incurs additional complexity for applications, and sometimes limits the expressiveness of the data models you can use.
4. Replication provides redundancy: if some nodes are unavailable, the data can still be served from the remaining nodes.
5. These are separate mechanisms, but they often go hand in hand, as illustrated in Figure II-1.

Chapter 6 | Quotes From Pages 167-212

1. Replication means keeping a copy of the same data on multiple machines that are connected via a network.
2. If the data that you're replicating does not change over time, then replication is easy: you just need to copy the data to every node once, and you're done.
3. Leader-based replication is not restricted to only databases:

More Book Summary
Scan to Download
Listen summary

distributed message brokers such as Kafka and RabbitMQ also use it.

4. In practice, if you enable synchronous replication on a database, it usually means that one of the followers is synchronous, and the others are asynchronous.
5. Replication turns out to be a remarkably tricky problem. It requires carefully thinking about concurrency and about all the things that can go wrong, and dealing with the consequences of those faults.

More Book Summary
Scan to Download
Listen summary

Download Bookey App to enjoy

1 Million+ Quotes

1000+ Book Summaries

Free Trial Available!

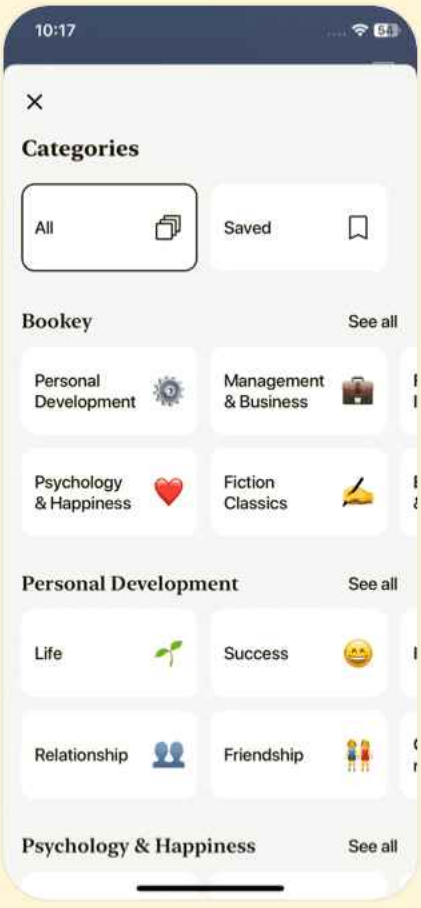
Scan to Download

Download on the App Store

GET IT ON Google Play







Chapter 7 | Quotes From Pages 213-234

1. The main reason for wanting to partition data is scalability.
2. If the partitioning is unfair, so that some partitions have more data or queries than others, we call it skewed. This makes the partitioning much less effective.
3. The simplest approach of avoiding hot spots would be to assign records to nodes randomly. That would distribute the data quite evenly across the nodes, but has a big disadvantage: when you're trying to read a particular item, you have no way of knowing which node it is on.
4. By design, every partition operates mostly independently — that's what allows a partitioned database to scale to multiple machines.
5. If your database only supports a key-value model, you might be tempted to implement a secondary index yourself by creating a value-to-document-ID mapping in application code.

Chapter 8 | Quotes From Pages 235-286

More Book Summary
Scan to Download
Listen summary

1. In the harsh reality of data systems, many things can go wrong: The database software or hardware may fail at any time, the application may crash at any time, interruptions in the network can unexpectedly cut off the application from the database.
2. For decades, transactions have been the mechanism of choice for simplifying these issues. A transaction is a way for an application to group several reads and writes together into a logical unit.
3. Transactions are not a law of nature; they were created with a purpose, namely, in order to simplify the programming model for applications accessing a database.
4. Although transactions seem straightforward at first glance, there are actually many subtle, but important details that come into play.
5. Even many popular relational database systems, which are usually considered ‘ACID’, use weak isolation, so they wouldn’t necessarily have prevented these bugs from

More Book Summary
Scan to Download
Listen summary

occurring.

6.That is not a new problem; it has been like this since the 1970s, when weak isolation levels were first introduced.

Chapter 9 | Quotes From Pages 287-332

1. In the end, our task as engineers is to build systems that do their job (i.e., meet the guarantees that users are expecting), in spite of everything going wrong.
2. Thus, if an internal fault occurs, we prefer a computer to crash completely, rather than return a wrong result, because wrong results are difficult and confusing to deal with.
3. In distributed systems, we try to build tolerance of partial failures into software, so that the system as a whole may continue functioning, even when some of its constituent parts are broken.
4. In distributed systems, suspicion, pessimism, and paranoia pay off.
5. To tolerate faults, the first step is to detect the fault, but even that is hard.

More Book Summary
Scan to Download
Listen summary

6. When you are writing software that runs on several computers, connected by a network, the situation is fundamentally different.
7. It is possible to give hard real-time response guarantees and bounded delay in networks, but doing so is very expensive and results in lower utilization of hardware resources.
8. If the error handling strategy consists of simply giving up, such a large system would never work.
9. Even in smaller systems, consisting of only a few nodes, it’s important to think about partial failure.
10. It is important to consider a wide range of possible faults,— even fairly unlikely ones,— and to artificially create such situations in your testing environment, to see what happens.

More Book Summary
Scan to Download
Listen summary

Download Bookey App to enjoy

1 Million+ Quotes

1000+ Book Summaries

Free Trial Available!

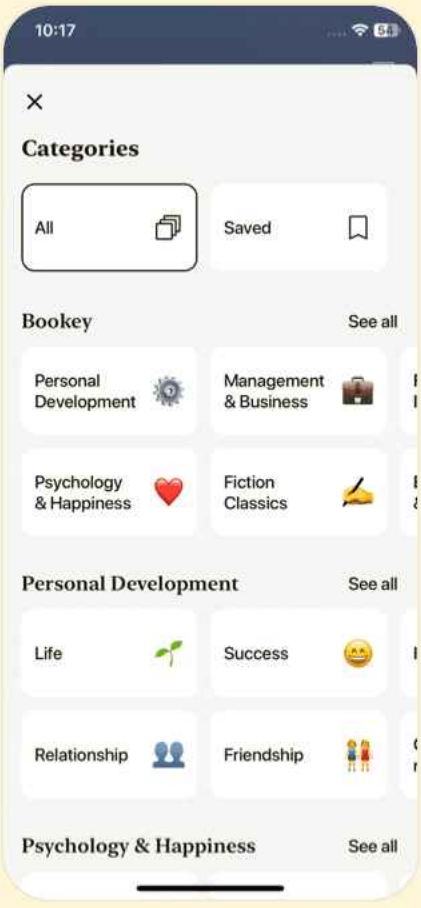
Scan to Download

Download on the App Store

GET IT ON Google Play







Chapter 10 | Quotes From Pages 333-396

1. The best way of building fault-tolerant systems is to find some general-purpose abstractions with useful guarantees, implement them once, and then let applications rely on those guarantees.
2. Eventual consistency is hard for application developers because it is so different from the behavior of variables in a normal single-threaded program.
3. The basic idea behind linearizability is simple: make a system appear as if there was only a single copy of the data.
4. The truth is defined by the majority.
5. Consensus is one of the most important and fundamental problems in distributed computing.
6. CAP theorem: in the presence of a network partition, a system can provide either consistency or availability, but not both.
7. In certificate-based systems, there's a need to either guarantee an agreed-upon state or be able to recover from

More Book Summary
Scan to Download
Listen summary

conflicts caused by diverging states.

Chapter 11 | Quotes From Pages 397-398

1. In reality, integrating disparate systems is one of the most important things that needs to be done in a non-trivial application.
2. The distinction between system of record and derived data system depends not on the tool, but on how you use it in your application.
3. By being clear about which data is derived from which other data, you can bring clarity to an otherwise confusing system architecture.

Chapter 12 | Quotes From Pages 399-446

1. A system cannot be successful if it is too strongly influenced by a single person. Once the initial design is complete and fairly robust, the real test begins as people with many different viewpoints undertake their own experiments."—Donald Knuth
2. In such online systems, whether it's a web browser

More Book Summary
Scan to Download
Listen summary

requesting a page, or a service calling a remote API, we generally assume that the request is triggered by a human user, and that the user is waiting for the response. They shouldn't have to wait too long for the response, so we pay a lot of attention to the response time of these systems.

3. Batch processing is a very old form of computing. Long before programmable digital computers were invented, punch card tabulating machines—such as the Hollerith machines used in the 1890 US Census—implemented a semi-mechanized form of batch processing to compute aggregate statistics from large inputs.
4. If you expect the output of one program to become the input to another program, that means those programs must use the same data format— in other words, a compatible interface.
5. Surprisingly, many data analyses can be done in a few minutes using some combination of awk, sed, grep, sort, uniq, and xargs, and they perform surprisingly well.

More Book Summary
Scan to Download
Listen summary

Download Bookey App to enjoy

1 Million+ Quotes

1000+ Book Summaries

Free Trial Available!

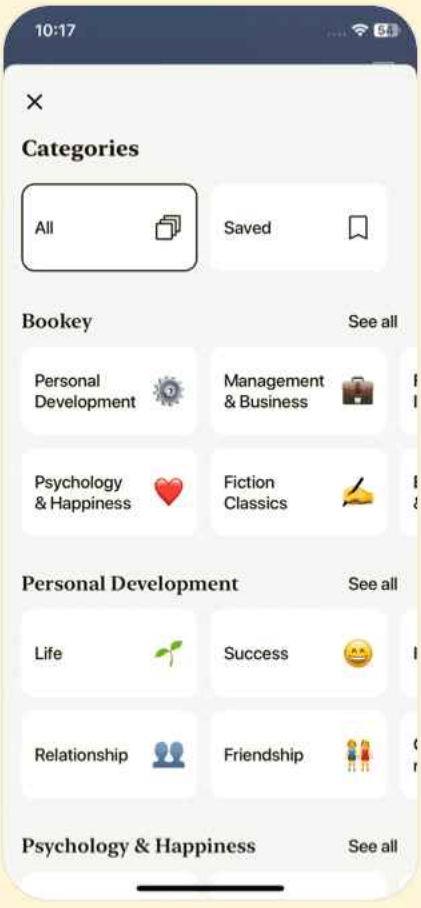
Scan to Download

Download on the App Store

GET IT ON Google Play







Chapter 13 | Quotes From Pages 447-491

1. A complex system that works is invariably found to have evolved from a simple system that works. The inverse proposition also appears to be true: A complex system designed from scratch never works and cannot be made to work." — John Gall, Systemantics (1975)
2. In general, a stream refers to data that is incrementally made available over time.
3. That is the idea behind stream processing.
4. To reduce the delay, we can run the processing more frequently — say, processing a second's worth of data at the end of every second — or even continuously, abandoning the fixed time-slices entirely, and simply processing every event as it happens.
5. The problem with daily batch processes is that changes in the input are only reflected in the output a day later, which is too slow for many impatient users.
6. When the input is a file (a sequence of bytes), the first

More Book Summary
Scan to Download
Listen summary

processing step is usually to parse it into a sequence of records.

7. An event usually contains a timestamp indicating when it happened.
8. If you consider the log of events to be your system of record, and any mutable state as being derived from it, it becomes easier to reason about the flow of data through a system.
9. Immutable events also capture more information than just the current state.
10. You can keep derived data systems such as search indexes, caches, and analytics systems continually up-to-date by consuming the log of changes and applying them to the derived system.

More Book Summary
Scan to Download
Listen summary

Download Bookey App to enjoy

1 Million+ Quotes

1000+ Book Summaries

Free Trial Available!

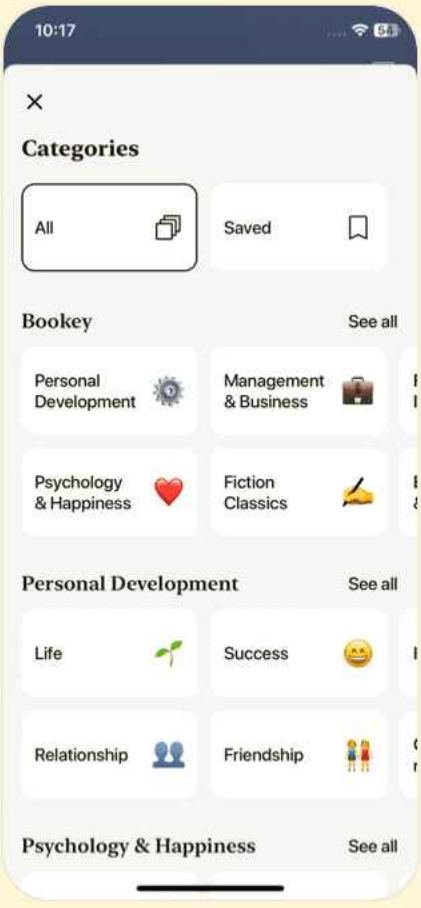
Scan to Download

Download on the App Store

GET IT ON Google Play







Designing Data-Intensive Applications Questions

View on Bookey Website

Chapter 1 | Reliable, Scalable and Maintainable Applications| Q&A

1.Question

What are the key concerns when designing data-intensive applications?

Answer:The key concerns include reliability, scalability, and maintainability. Reliability ensures the system continues to operate correctly despite faults; scalability involves the ability to handle increased loads gracefully; maintainability ensures the system can be adapted and improved over time without excessive cost or complexity.

2.Question

How do reliability, scalability, and maintainability interact with each other in software systems?

Answer:These three principles are interrelated; a highly reliable system may also be scalable if designed properly, but

More Book Summary

Scan to Download

Listen summary

achieving both often requires careful consideration of the architecture. Additionally, maintainability can be impacted by how reliably the system can operate, thus by designing for maintainability, developers can also enhance reliability and scalability.

3.Question

Why is it important to consider faults in hardware, software, and human operators?

Answer:Understanding that faults can arise from hardware failures, software bugs, or human errors is critical for creating fault-tolerant systems. Each type of fault presents unique challenges; for instance, hardware faults are often random while software bugs can be systematic. Designing for these faults ensures the application remains stable and serves users effectively.

4.Question

What strategies can be employed to enhance system reliability?

Answer:Strategies include building redundancy into

More Book Summary
Scan to Download
Listen summary

hardware components, implementing software fault-tolerance techniques, thorough testing to catch bugs early, and creating systems that can recover easily from errors, making it fast to revert unwanted changes.

5.Question

How can an organization ensure the scalability of its applications as it grows?

Answer:Organizations can ensure scalability by understanding and describing their current system load quantitatively, and then planning to distribute that load effectively across more resources, whether by scaling up (enhancing the capability of existing machines) or scaling out (adding more machines).

6.Question

What is the role of simplicity in maintainability?

Answer:Simplicity is crucial in maintaining software systems as it reduces complexity, making it easier for new engineers to understand and work with the system. Adopting good abstractions can help in hiding unnecessary complexity,

More Book Summary
Scan to Download
Listen summary

thereby allowing the team to focus on improving and adapting the system rather than getting bogged down by its intricacies.

7.Question

Can you provide an example of a common scalability challenge?

Answer:Consider Twitter's challenge with handling tweets. As the volume of tweets grew significantly, the system initially struggled to process home timeline requests efficiently. The company had to rethink its architecture to implement caching for each user's timeline, ensuring faster read responses by saving computations ahead of time.

8.Question

What are the recommended practices for ensuring human errors do not compromise system reliability?

Answer:To mitigate human errors, best practices include designing user interfaces that discourage mistakes, providing sandbox environments for safe experimentation, thorough testing of workflows, and implementing comprehensive

More Book Summary
Scan to Download
Listen summary

monitoring systems to catch errors as they happen.

9.Question

How can organizations measure and address performance during high-load scenarios?

Answer:Organizations can measure performance using metrics like response times and throughput, focusing on percentile values rather than averages to identify slow requests. They can then simulate increased loads to see how the system behaves and apply scaling techniques to ensure performance remains within acceptable limits.

10.Question

Why is evolvability important in software systems?

Answer:Evolvability is essential because requirements change over time, and systems must be adaptable to new use cases, features, or regulations. Designing for evolvability makes it easier to implement changes without significant redesign, thus ensuring longevity and relevance of the software application.

Chapter 2 | Data Models and Query Languages| Q&A

More Book Summary
Scan to Download
Listen summary

1.Question

Why are data models considered the most important part of developing software?

Answer:Data models provide a foundational structure for how applications process and manage data, dictating how developers interact with the data, the complexity that is hidden from them, and how data is represented across various systems. The choice of data model profoundly influences the software's functionality and performance.

2.Question

What is the significance of selecting an appropriate data model for an application?

Answer:Selecting a suitable data model is crucial because it directly impacts how easily an application can evolve, how efficiently it performs operations, and how well it meets user requirements. The right model aligns with the application's specific use cases, whether transactional, analytical, or for real-time processing.

More Book Summary

Scan to Download

Listen summary

3.Question

How do the examples of relational models and document models differ in handling data?

Answer:Relational models typically involve structured data that is stored across multiple tables with foreign keys, necessitating complex joins for data retrieval. In contrast, document models like JSON or XML store related information together in single documents, leading to simpler retrieval at the expense of complicated updates when relationships change.

4.Question

What challenges arise from the 'impedance mismatch' between object-oriented programming and relational databases?

Answer:The impedance mismatch refers to the difficulties encountered when translating data between the object-oriented paradigms used in application programming and the relational model's tabular structures. This often requires additional layers of abstraction or Object-Relational Mapping (ORM) frameworks that can add complexity and

More Book Summary





Scan to Download

Listen summary

overhead, hindering performance.

5.Question

What are the main driving forces behind the rise of NoSQL databases?

Answer:The push for NoSQL databases stems from needs for greater scalability, flexibility in data structures, varied querying capabilities, and frustration with the rigidity of relational schemas. Organizations increasingly demand databases that can adapt to changing data requirements rapidly without extensive reconfiguration.

6.Question

What are the pros and cons of using schema-on-read versus schema-on-write in database design?

Answer:Schema-on-read provides flexibility by allowing application developers to interpret data upon reading, which is beneficial for evolving data structures. However, it lacks enforced validation at write time, risking inconsistencies. In contrast, schema-on-write enforces data integrity and consistency but can require complex migrations and

More Book Summary
Scan to Download
Listen summary

downtime to alter schemas.

7.Question

How does the performance advantage of document databases manifest in typical application use cases?

Answer:Document databases like MongoDB allow for efficient retrieval of entire records without needing to look up related items across tables, as all relevant data is stored together. This locality reduces the number of disk reads required, speeding up read operations, especially important in applications that frequently load and display full records.

8.Question

In what scenarios are graph databases more advantageous than relational databases?

Answer:Graph databases are particularly beneficial for applications with complex many-to-many relationships where entities are heavily interconnected, such as social networks or recommendation systems. Their structure allows for more straightforward modeling of relationships and efficient querying of graph-like data.

More Book Summary





Scan to Download

Listen summary

9.Question

What role does a query language play in the effectiveness of a data model?

Answer:A query language shapes how developers interact with the data model, often determining ease of use, expressiveness, and performance. Declarative languages like SQL allow for more straightforward querying and optimizations by the database engine, while imperative languages require detailed control of operations, often leading to more complex and less efficient code.

10.Question

How can choosing the wrong data model impact an application's development and maintenance?

Answer:Using an unsuitable data model can lead to inefficiencies, increased complexity in data handling, and difficulties in maintaining consistency across the application. This can result in slower performance, more challenging debugging processes, and higher costs associated with development and ongoing support.

More Book Summary





Scan to Download

Listen summary

Chapter 3 | Storage and Retrieval| Q&A

1.Question

What are the two fundamental responsibilities of a database according to Chapter 3?

Answer:A database must store data when it is provided, and retrieve that data when requested later.

2.Question

Why is it important for application developers to understand how databases handle storage and retrieval?

Answer:Understanding the internal handling of storage and retrieval allows developers to choose an appropriate storage engine for their application, ensuring optimal performance for their specific workload.

3.Question

What is the primary difference between transactional and analytical workloads as discussed in the chapter?

Answer:Transactional workloads (OLTP) involve accessing a small number of records per query with a focus on low-latency interactions, while analytical workloads (OLAP)

More Book Summary

Scan to Download

Listen summary

involve scanning large datasets for aggregate queries.

4.Question

Describe the simple implementation of a key-value store presented in the chapter. How does it work?

Answer:It consists of two Bash functions, db_set() and db_get(). db_set() appends a key-value pair to a text file, while db_get() searches the file for the most recent value associated with the key using grep and tail commands.

5.Question

What performance issue arises with the db_get function when the database grows large?

Answer:The performance degrades significantly because db_get performs a linear scan of the entire database file, leading to a lookup cost of O(n), where n is the number of records.

6.Question

What is indexing in the context of databases, and why is it necessary?

Answer:Indexing is an additional data structure that helps locate specific data efficiently, reducing the lookup time for

More Book Summary
Scan to Download
Listen summary

records and improving query performance compared to naive search methods.

7.Question

Explain the role of hashing in the db_set and db_get example. How does it improve performance?

Answer:Hashing allows quick access to values based on keys. By using an in-memory hash map for the offsets of key-value pairs in the file, lookups can be done in O(1) rather than O(n), making the retrieval much faster.

8.Question

What are SSTables, and how do they differ from log-based storage?

Answer:SSTables (Sorted String Tables) are a form of storage that keeps key-value pairs sorted by key within a segment. Unlike log-based storage, which appends to non-sorted segments, SSTables enable efficient merging and range queries.

9.Question

What are some key considerations when choosing to use indexes in a database?

More Book Summary
Scan to Download
Listen summary

Answer:When selecting indexes, one must balance the performance benefits of speeding up reads against the overhead added to write operations, since maintaining indexes incurs additional write costs.

10.Question

How does a B-tree data structure support database indexing?

Answer:B-trees allow for efficient searching, insertion, and deletion by organizing data in a tree structure where each node contains multiple keys. This structure ensures that data remains balanced, allowing for logarithmic search time.

11.Question

Discuss the characteristics of column-oriented storage and its advantages for analytical queries.

Answer:Column-oriented storage stores data in columns rather than rows, enabling the retrieval of only the necessary columns in a query. This improves I/O efficiency, allows for better data compression, and enhances the performance of aggregate queries.

More Book Summary





Scan to Download

Listen summary

12.Question

What are data cubes, and how do they benefit analytics in data warehouses?

Answer:Data cubes are multi-dimensional arrays of data aggregates stored for quick access. They allow for rapid querying of summarized data across various dimensions, enabling efficient analytical queries without scanning raw detail data.

13.Question

Explain the importance of ETL in the context of using data warehouses.

Answer:ETL (Extract, Transform, Load) is critical as it enables the aggregation of data from multiple OLTP systems into a centralized data warehouse where it is structured and optimized for analysis without affecting the performance of operational systems.

14.Question

What future considerations are mentioned regarding in-memory databases?

Answer:As memory prices decrease, the feasibility of

More Book Summary
Scan to Download
Listen summary

maintaining larger datasets entirely in memory is increasing, prompting innovations in database architectures that can efficiently manage datasets that exceed physical memory capacity.

More Book Summary
Scan to Download
Listen summary

Ad

Scan to Download



Why Bookey is must have App for Book Lovers

30min Content
The deeper and clearer interpretation we provide, the better grasp of each title you have.

Text and Audio format
Absorb knowledge even in fragmented time.

Quiz
Check whether you have mastered what you just learned.

And more
Multiple Voices & fonts, Mind Map, Quotes, IdeaClips...

Free Trial with Bookey

Chapter 4 | Encoding and Evolution| Q&A

1.Question

What is the fundamental idea of evolvability in data-intensive applications?

Answer:Evolvability is the design principle that enables systems to easily adapt to changes, facilitating the addition or modification of application features as user requirements evolve or business circumstances shift.

2.Question

How do relational databases handle changes in data schema?

Answer:Relational databases enforce a single schema that must be adhered to, and any changes require schema migrations, like ALTER statements, to update the existing schema.

3.Question

What are the advantages of schema-on-read databases compared to traditional schema databases?

Answer:Schema-on-read databases, or schemaless databases,

More Book Summary





Scan to Download

Listen summary

allow data to be stored in various formats without enforcing a rigid schema, enabling them to accommodate a mix of old and new data formats more efficiently.

4.Question

What compatibility is required during rolling upgrades of software applications?

Answer:Both backward compatibility (new code can read old data) and forward compatibility (old code can read new data) are crucial for ensuring smooth transitions during rolling upgrades.

5.Question

What role do encoding formats like JSON and XML play in data evolution?

Answer:Encoding formats like JSON and XML provide human-readable data interchange formats that can support some level of schema changes, but their compatibility can be limited depending on how they are utilized.

6.Question

What is the significance of binary encoding formats like Protocol Buffers in schema evolution?

More Book Summary





Scan to Download

Listen summary

Answer:Binary encoding formats require a schema that allows for compact and efficient encoding, ensuring defined forward and backward compatibility while also serving as documentation for data structures.

7.Question

Why is preserving unknown fields during updates important in data systems?

Answer:Preserving unknown fields ensures that data written by newer versions of applications is not lost when older versions read and update records, thus maintaining data integrity.

8.Question

What are the advantages of using Apache Avro for data storage in evolving applications?

Answer:Apache Avro allows for flexible schema evolution, enabling the addition and removal of fields while maintaining compatibility between different schema versions, as well as enabling efficient binary encoding.

9.Question

How do actual data flows differ between REST APIs and

More Book Summary
Scan to Download
Listen summary

traditional RPC calls?

Answer:REST APIs handle requests and responses over HTTP, which can include various data formats, whereas RPC attempts to make remote calls resemble local function calls, often abstracting the network complexity.

10.Question

What challenges arise with versioning in web services APIs?

Answer:APIs need to maintain backward compatibility for old clients while accommodating new functionalities, often leading to the need for multiple versions to exist simultaneously.

11.Question

In what ways does evolvability enhance application deployment and upgrade strategies?

Answer:Evolvability allows for frequent deployments through rolling upgrades, where components can be updated independently without service downtime, improving operational efficiency and reducing the risk associated with

More Book Summary

Scan to Download

Listen summary

large-scale changes.

Chapter 5 | Part II. Distributed Data| Q&A

1.Question

What are some key reasons for distributing a database across multiple machines?

Answer:1. Scalability: Distributing the workload allows for handling larger data volumes, read loads, or write loads beyond a single machine's capacity. 2. Fault tolerance/high availability: It offers redundancy, ensuring the application continues to work even if one or several machines fail. 3. Latency reduction: Having servers in various geographical locations allows user requests to be served from much closer data centers, minimizing wait times.

2.Question

What does a shared-nothing architecture entail, and why is it popular?

Answer:A shared-nothing architecture consists of multiple independent nodes, each managing its own CPU, RAM, and

More Book Summary
Scan to Download
Listen summary

disks. This setup is popular because it avoids the bottlenecks associated with shared resources, and it allows for better price/performance optimization. As there's no special hardware required, it remains flexible and scalable, making it feasible even for small companies to operate across multiple regions.

3.Question

What are the trade-offs involved when using a distributed system?

Answer:In a distributed system, while it provides scalability and fault tolerance, it introduces complexity in application development. Developers need to be mindful of data consistency, potential latencies in communication between nodes, and the limitations in expressiveness of data models. Sometimes, simple applications can outperform complex distributed systems, emphasizing the need for careful architectural decisions.

4.Question

How does replication differ from partitioning in data distribution?

More Book Summary
Scan to Download
Listen summary

Answer:Replication involves creating multiple copies of the same data across different nodes for redundancy and performance improvement. Partitioning, also known as sharding, divides a large database into smaller subsets, assigning each to different nodes. While replication offers backup and can enhance retrieval times, partitioning optimizes storage and manages larger datasets by spreading the load.

5.Question

What fundamental limitations should one understand when it comes to distributed systems?

Answer:Distributed systems face inherent challenges such as network partitions, data consistency issues, and the complexities of managing state across multiple nodes. Understanding these limitations is crucial for developing robust applications that can function reliably under various failure scenarios.

Chapter 6 | Replication| Q&A

1.Question

More Book Summary
Scan to Download
Listen summary

What is replication in data-intensive applications, and why is it important?

Answer:Replication is the process of keeping a copy of the same data on multiple machines connected via a network. It is important because it enables geographic distribution of data to reduce latency for users, increases availability of the system by allowing it to continue functioning even if some parts fail, and allows scaling out the number of machines that can handle read queries, thereby enhancing read throughput.

2.Question

What are the three popular algorithms for replicating data changes?

Answer:The three popular algorithms for replicating data changes are single-leader (master-slave), multi-leader (master-master), and leaderless replication. Each of these has its advantages and disadvantages based on factors such as consistency guarantees and fault tolerance.

More Book Summary





Scan to Download

Listen summary

3.Question

What are the trade-offs between synchronous and asynchronous replication?

Answer:Synchronous replication guarantees that all replicas are up-to-date before confirming a write operation, enhancing durability. However, it may block writes if a follower does not respond. Asynchronous replication allows writes to proceed without waiting for followers, which can improve performance but risks losing updates if the leader fails before replication completes.

4.Question

What issues can arise from replication lag, and how can they affect user experience?

Answer:Replication lag can cause users to see outdated data if they read from followers that are behind the leader. This can lead to inconsistencies, such as a user viewing their recently submitted data as non-existent, leading to confusion and dissatisfaction. Implementing read-after-write consistency mechanisms can mitigate this issue.

More Book Summary





Scan to Download

Listen summary

5.Question

What is eventual consistency, and why is it significant in the context of distributed databases?

Answer:Eventual consistency is the guarantee that if no new updates are made to a given piece of data, eventually all accesses to that data will return the last updated value. This is significant because it provides a simpler model for distributed systems to ensure availability while tolerating temporary inconsistencies.

6.Question

How can multi-leader replication offer advantages in multi-datacenter operations?

Answer:In multi-leader replication, each datacenter can process local writes without relying solely on a central leader, which reduces latency and improves performance. It also allows each datacenter to operate independently in case of outages, as writes can continue in other datacenters while the affected one is recovering.

7.Question

What is a quorum in the context of leaderless replication,

More Book Summary
Scan to Download
Listen summary

and how does it relate to data durability?

Answer:A quorum in leaderless replication refers to the minimum number of nodes that must acknowledge a write (denoted by 'w') before it is considered successful. It ensures that the write is sufficiently durable, as it must be present on a majority of replicas to prevent data loss in case of failures.

8.Question

How do version vectors aid in detecting concurrent writes in leaderless systems?

Answer:Version vectors track the version number of each write operation per key and per replica, allowing the system to recognize which values are concurrent. This helps in resolving conflicts by merging concurrent writes instead of simply overwriting values, thus maintaining data integrity.

9.Question

What complexities arise from implementing conflict resolution in multi-leader setups?

Answer:In multi-leader setups, complexities include detecting conflicting writes when multiple replicas accept

More Book Summary

Scan to Download

Listen summary

concurrent updates and managing the merge of conflicting changes. The lack of a single authorative order for writes makes it challenging to ensure consistency across replicas, requiring robust conflict resolution strategies.

10.Question

Why is measuring replication lag vital for operation management in distributed databases?

Answer:Monitoring replication lag is crucial because it informs operators when data consistency issues may arise. If lag increases significantly, it can indicate underlying problems such as network issues or overloaded nodes, which require immediate attention to prevent user-facing consistency problems.

More Book Summary

Scan to Download

Listen summary

Ad

Scan to Download

App Store
Editors' Choice

22k 5 star review

Positive feedback

Sara Scholz

tes after each book summary
erstanding but also make the
and engaging. Bookey has
ding for me.

Fantastic!!!

Masood El Toure

I'm amazed by the variety of books and languages Bookey supports. It's not just an app, it's a gateway to global knowledge. Plus, earning points for charity is a big plus!

Fi

Ab
bo
to
m

José Botín

ding habit
p's design
ual growth

Love it!

Wonnie Tappkx

Bookey offers me time to go through the important parts of a book. It also gives me enough idea whether or not I should purchase the whole book version or not! It is easy to use!

Time saver!

Bookey is my go-to app for
summaries are concise, ins
curated. It's like having acc
right at my fingertips!

Awesome app!

Rahul Malviya

I love audiobooks but don't always have time to listen to the entire book! bookey allows me to get a summary of the highlights of the book I'm interested in!!! What a great concept !!!highly recommended!

Beautiful App

Alex Walk

This app is a lifesaver for book lovers with busy schedules. The summaries are spot on, and the mind maps help reinforce wh I've learned. Highly recommend!

Free Trial with Bookey

Chapter 7 | Partitioning| Q&A

1.Question

What is the main reason for partitioning data in databases?

Answer:The primary reason for partitioning data is scalability. Partitioning enables large datasets to be distributed across multiple nodes, allowing for an increase in storage and query throughput by leveraging the resources of many machines instead of relying on a single machine.

2.Question

How can partitions be defined in relation to data records?

Answer:Partitions are typically defined so that each piece of data (record, row, or document) belongs to exactly one partition. The mechanism for this can vary, but the goal is to ensure that the data and query load is spread evenly across partitions.

3.Question

What is a hot spot in the context of partitioning, and how can it be avoided?

More Book Summary





Scan to Download

Listen summary

Answer:A hot spot occurs when one partition receives disproportionately high loads while others are idle, leading to performance bottlenecks. To avoid this, effective partitioning strategies must be employed, such as randomizing the assignment of records to nodes, or using a hash function to distribute load more evenly.

4.Question

Can you describe the difference between key-range partitioning and hash partitioning?

Answer:Key-range partitioning involves assigning continuous ranges of keys to each partition, allowing for efficient range queries but potentially leading to hot spots. In contrast, hash partitioning uses a hash function to determine the partition for each key, which helps distribute load evenly but sacrifices the ability to perform efficient range queries because the key order is lost.

5.Question

What are some challenges involved in rebalancing partitions?

More Book Summary

Scan to Download

Listen summary

Answer:Rebalancing involves redistributing data across partitions to account for changes in query workload or data volume. It must be done carefully due to its potential to overload the network and existing nodes, especially if it occurs in response to system failures or load imbalances.

6.Question

What solution can be implemented to handle dynamic changes in partitions as data grows?

Answer:Dynamic partitioning can be used, where partitions are created or split automatically as data reaches a certain size. This allows the database architecture to adapt to changing data volumes without requiring constant manual intervention.

7.Question

How does request routing function in a partitioned database architecture?

Answer:Request routing determines which node a client should connect to when a request is made for a particular key. This can be managed through different strategies,

More Book Summary
Scan to Download
Listen summary

including allowing clients to connect randomly to any node, using a dedicated routing tier to forward requests, or having clients manage their own knowledge of partition assignments.

8.Question

What is the significance of secondary indexes in partitioned databases?

Answer:Secondary indexes provide a way to query records based on non-primary key values. However, partitioning strategies must be adjusted to account for secondary indexes, as they may require additional complexity to ensure efficient access without sacrificing the performance benefits of partitioning.

9.Question

How does consistent hashing improve partitioning?

Answer:Consistent hashing minimizes the data moved between partitions when scaling the cluster, making it more efficient to adapt to adding or removing nodes without redistributing all keys, as it only requires transferring the

More Book Summary
Scan to Download
Listen summary

partitions that have changed.

Chapter 8 | Transactions| Q&A

1.Question

What are the key advantages of using transactions in database applications?

Answer:Transactions abstract away concurrency issues and potential hardware/software faults, allowing developers to handle errors as simple transaction aborts. They simplify programming by ensuring consistency and integrity of operations.

2.Question

Why might some applications not require any form of transactions?

Answer:Applications with very simple access patterns, such as those involving only single record reads and writes, may function correctly without the complexity of transactions.

3.Question

How does the 'ACID' property enhance the reliability of transactions?

Answer:ACID stands for Atomicity, Consistency, Isolation,

More Book Summary
Scan to Download
Listen summary

and Durability, ensuring that transactions are completed fully or not at all, maintain data integrity even in the case of errors, handle concurrent operations without interfering with each other, and ensure that once data is committed, it is permanently stored.

4.Question

What are the drawbacks of relying on weak isolation levels in transactions?

Answer:Weak isolation levels can lead to concurrency problems such as read skew, lost updates, and write skew, which in turn may result in inconsistencies and bugs that are hard to debug.

5.Question

What is the meaning of 'write skew' and how can it occur in concurrent transactions?

Answer:Write skew occurs when two transactions read the same data and both decide to write without acknowledging the other's changes, leading to potential inconsistencies. For example, if two doctors both go off call simultaneously

More Book Summary
Scan to Download
Listen summary

without checking that at least one remains, it could leave no doctor on call.

6.Question

What are some methods to prevent lost updates when multiple transactions are involved?

Answer:Atomic update operations, explicit locking, and automatic conflict detection help prevent lost updates by ensuring that updates integrate all relevant changes without clobbering one another.

7.Question

How does snapshot isolation prevent many forms of concurrency issues?

Answer:Snapshot isolation allows each transaction to operate on a consistent snapshot of the database at the time it starts, thus avoiding issues like dirty reads or non-repeatable reads, although it might still allow certain anomalies such as write skew.

8.Question

What is the significance of the two-phase locking protocol and its downside?

More Book Summary
Scan to Download
Listen summary

Answer:Two-phase locking (2PL) is crucial for ensuring serializability and preventing race conditions, but it can significantly reduce performance due to increased contention among transactions and potential latency issues.

9.Question

Explain how serializable snapshot isolation (SSI) differs from traditional locking mechanisms. What are its benefits?

Answer:Serializable snapshot isolation allows for concurrent transactions without locking resources, improving performance by minimizing blocking. It checks for consistency only at the commit point, reducing the overall coordination overhead.

10.Question

What are some inherent challenges in implementing transactions in distributed database systems?

Answer:Distributed transactions face complications in managing consistency across multiple nodes, potential network failures, and the need for coordination between distributed components, making them difficult to implement

More Book Summary

Scan to Download

Listen summary

and maintain.

11.Question

In what ways has the perception of transactions evolved with the rise of NoSQL databases?

Answer:Many NoSQL systems have abandoned traditional transaction mechanisms to prioritize scalability and performance, leading to applications that either manage their own consistency or accept eventual consistency, complicating error handling.

Chapter 9 | The Trouble with Distributed Systems| Q&A

1.Question

What is the main challenge in designing distributed systems?

Answer:The main challenge in designing distributed systems lies in managing the unpredictability of network communication, which includes issues like packet loss, delays, and partial failures. Unlike single-computer environments, where behavior is deterministic, distributed systems must contend with

More Book Summary
Scan to Download
Listen summary

the fact that any operation involving multiple nodes may fail or succeed unpredictably due to these varying conditions.

2.Question

Why is fault tolerance critical in distributed systems?

Answer:Fault tolerance is critical because distributed systems often operate with unreliable components, meaning individual nodes may fail without warning. To meet user expectations and maintain functionality, systems must be capable of continuing operations despite these failures.

3.Question

How does network unreliability affect distributed systems?

Answer:Network unreliability can lead to situations where messages are lost, delayed, or not acknowledged. This makes it essential for systems to handle such failures gracefully. Timeout mechanisms and quorum-based decision-making are strategies used to manage these uncertainties, but they introduce complexity to the system design.

More Book Summary
Scan to Download
Listen summary

4.Question

What role do clocks play in distributed systems?

Answer:Clocks are essential for tasks like event ordering and timeout handling. However, differences in clock synchronization can lead to inconsistencies in how events are perceived across nodes, complicating tasks such as coordinating operations or detecting failures.

5.Question

What are the differences between synchronous and asynchronous models?

Answer:In the synchronous model, bounded delays and pauses are assumed, allowing for predictable communication. The asynchronous model, however, does not permit any time assumptions, leading to potentially unbounded delays. This difference significantly affects how algorithms are designed and implemented.

6.Question

How can distributed systems handle the problem of clients mistakenly believing they are still 'alive'?

Answer:To mitigate this, mechanisms such as fencing tokens

More Book Summary
Scan to Download
Listen summary

are employed. These tokens ensure that even if a client believes it still holds a lock or lease, it must provide a token reflecting its last known state, preventing conflicting writes from different clients.

7.Question

What is a Byzantine fault and why is it significant in designing distributed algorithms?

Answer:A Byzantine fault occurs when a node behaves arbitrarily due to a failure or malicious intent, potentially lying about its state. This concept is crucial for ensuring that distributed algorithms can reach consensus and maintain correct operations even in the presence of such deception.

8.Question

Why is it important to define safety and liveness properties in distributed algorithms?

Answer:Defining safety and liveness properties helps clarify what must be guaranteed by the algorithm. Safety ensures that nothing bad occurs (e.g., no wrong results), while liveness guarantees that something good will eventually

More Book Summary
Scan to Download
Listen summary

happen (e.g., successful operations). Distinguishing these helps manage the complexities of various system states.

9.Question

What impact do process pauses have on distributed systems?

Answer:Process pauses can lead to nodes being declared dead by others if they fail to respond within expected time frames. This might cause operational inconsistency, as the paused node can resume and continue functioning without realizing it was deemed inactive.

10.Question

How do quorum mechanisms help solve issues in distributed systems?

Answer:Quorum mechanisms allow a majority of nodes to make decisions, which helps to ensure consistent and correct behavior even when some nodes are down or experiencing faults. This decentralized decision-making is crucial in mitigating the effects of partial failures.

More Book Summary
Scan to Download
Listen summary

Ad

Scan to Download



Read, Share, Empower

Finish Your Reading Challenge, Donate Books to African Children.

The Concept

BOOKS FOR AFRICA × ×

This book donation activity is rolling out together with Books For Africa. We release this project because we share the same belief as BFA: For many children in Africa, the gift of books truly is a gift of hope.

The Rule

Earn 100 points → **Redeem a book** → **Donate to Africa**

Your learning not only brings knowledge but also allows you to earn points for charitable causes! For every 100 points you earn, a book will be donated to Africa.

Free Trial with Bookey

Chapter 10 | Consistency and Consensus| Q&A

1.Question

What is the most straightforward method for handling faults in distributed systems according to Chapter 9?

Answer:The simplest way to handle faults is to allow the entire service to fail and show the user an error message. However, this is often unacceptable, leading to the necessity for fault-tolerant designs.

2.Question

How does consensus contribute to the functioning of a fault-tolerant distributed system?

Answer:Consensus allows all nodes in the system to agree on a certain state or decision, which is crucial in environments prone to failures, ensuring that the system remains coherent and maintains data integrity.

3.Question

What is the relationship between consensus and split-brain scenarios in distributed systems?

Answer:A split-brain situation occurs when two nodes believe they are the leader due to a failure in consensus,

More Book Summary

Scan to Download

Listen summary

causing potential data divergence. Correct implementations of consensus help avoid such conflicts by ensuring a single leader at any time.

4.Question

What guarantees does eventual consistency provide in replicated databases?

Answer:Eventual consistency guarantees that if no new updates are made to a given piece of data, eventually all accesses to that data will return the last updated value. However, it does not specify when this convergence will occur.

5.Question

How does linearizability differ from eventual consistency?

Answer:Linearizability offers a stronger guarantee by ensuring that once a value is written, any subsequent reads will reflect that update immediately, appearing as though only one copy of the data exists, while eventual consistency may allow for stale reads.

More Book Summary
Scan to Download
Listen summary

6.Question

What role does leader election play in distributed systems, and how is it related to consensus?

Answer:Leader election is a process of reaching a consensus on which node will act as the leader. This ensures that the system has a point of control for coordination, and the correctness of the election process is fundamental to maintaining consistency.

7.Question

What are some limitations of using two-phase commit (2PC) for distributed transactions?

Answer:2PC may block if the coordinator fails, leading to in-doubt transactions. Additionally, it lacks fault tolerance against network partitions, which could result in uncertain transaction outcomes.

8.Question

How can consensus algorithms be used to achieve total order broadcast in distributed systems?

Answer:Consensus algorithms can ensure that messages are delivered in a specific order across nodes, which is essential

More Book Summary
Scan to Download
Listen summary

for keeping replicas consistent. Each consensus decision can be thought of as an ordered delivery of a message.

9.Question

Why is it important to define a uniqueness constraint in distributed systems, and how does consensus help with this?

Answer:Uniqueness constraints prevent duplicate entries (like usernames) in databases. Consensus helps decide which entry should prevail when conflicts occur, thereby ensuring that the database maintains its integrity.

10.Question

What are the key properties that consensus algorithms must satisfy?

Answer:Consensus algorithms must satisfy uniform agreement (no two nodes decide differently), integrity (no node decides twice), validity (if a node decides value v, it had to have been proposed), and termination (every non-faulty node eventually decides).

11.Question

How does the CAP theorem relate to the choice between

More Book Summary
Scan to Download
Listen summary

consistency and availability in distributed systems?

Answer:The CAP theorem states that a distributed system can only guarantee two out of three properties: Consistency, Availability, and Partition tolerance. When a network partition occurs, a decision must be made to sacrifice either consistency or availability.

12.Question

How do systems like ZooKeeper and etcd utilize consensus algorithms?

Answer:Systems like ZooKeeper and etcd implement consensus algorithms to provide services like distributed locking and leader election, enabling applications to achieve coordination and maintain consistency across distributed nodes.

13.Question

What challenges do consensus algorithms face in practical implementations?

Answer:Consensus algorithms often deal with challenges like network delays, node failures, and the need for non-blocking

More Book Summary

Scan to Download

Listen summary

progress. They have to balance the requirement for strong consistency with the performance impacts of reaching consensus.

14.Question

Why is it important to have a fault tolerance strategy in distributed systems?

Answer:A fault tolerance strategy is essential in distributed systems to ensure reliability and availability, allowing the system to continue functioning correctly even if certain components fail.

Chapter 11 | Part III. Derived Data| Q&A

1.Question

What are the key differences between systems of record and derived data systems?

Answer:Systems of record hold the authoritative version of your data, representing facts exactly once and serving as a source of truth. Derived data systems, on the other hand, take existing data and transform it for various purposes, such as

More Book Summary
Scan to Download
Listen summary

performance optimization or analysis. If derived data is lost, it can be reconstructed from the original source.

2.Question

Why is the integration of multiple data systems important in application architecture?

Answer:Integrating multiple data systems is crucial because a single database often cannot meet all access and processing needs of a complex application. By combining different datastores, caches, and analytics systems, applications can be more flexible and efficient in how they handle data.

3.Question

How does derived data improve performance in applications?

Answer:Derived data, such as cached information or materialized views, often duplicates existing information but allows for faster read queries by providing quick access to processed data. This denormalization is essential in scenarios like recommendation systems where real-time performance is

More Book Summary
Scan to Download
Listen summary

critical.

4.Question

What is the significance of distinguishing between systems of record and derived data systems in architecture?

Answer:This distinction provides clarity in understanding data flow through the system. It explicitly delineates which parts of the system have specific inputs and outputs, and how they interdepend on each other, ultimately aiding in the design and maintenance of complex applications.

5.Question

What underlying principle does Part III of the book aim to convey regarding applications?

Answer:Part III focuses on the integration of various systems into a coherent application architecture, emphasizing the importance of understanding and managing the relationships between different data systems and their models to build robust and efficient applications.

6.Question

How do batch-oriented dataflow systems like MapReduce relate to the concepts of derived data and multi-system

More Book Summary
Scan to Download
Listen summary

integration?

Answer:Batch-oriented dataflow systems like MapReduce provide principles for processing and transforming data efficiently, which can be applied to stream processing in real-time scenarios. These concepts remain relevant when integrating multiple data systems to ensure seamless data flow and transformation.

7.Question

What challenges do vendors face when claiming their product can satisfy all application needs?

Answer:Vendors may overlook the complexity in integrating disparate systems. In reality, each application often requires a tailored mix of data systems that can handle various access patterns and data models, making a one-size-fits-all solution inadequate.

8.Question

How can clarity in data usage enhance application design?

Answer:By understanding which data is derived from which

More Book Summary

Scan to Download

Listen summary

sources and organizing the architecture accordingly, developers can simplify the application design process, making it easier to manage, scale, and maintain.

9.Question

In what ways can derived data be beneficial despite being redundant?

Answer:Derived data can significantly improve application performance by providing faster access to frequently queried information. It allows users to analyze data from multiple perspectives without having to fetch and process the data from the original sources every time.

10.Question

Why might traditional databases and query languages not inherently fit into the categories of system of record or derived data system?

Answer:Databases and query languages are tools that can be used in various ways. Their classification depends on how they are employed within a specific application context, rather than their inherent capabilities.

More Book Summary

Scan to Download

Listen summary

Chapter 12 | Batch Processing| Q&A

1.Question

What are the three types of data processing systems identified in the text?

Answer:The three types of data processing systems are: 1. Services (online systems), which respond to requests in a timely manner. 2. Batch processing systems (offline systems), which process large amounts of data at scheduled intervals without user interaction. 3. Stream processing systems (near-real-time systems), which handle data shortly after it is generated.

2.Question

How does batch processing differ from online processing in terms of performance metrics?

Answer:Batch processing primarily focuses on throughput, which measures the time taken to process a large set of data, while online processing emphasizes response time, aimed at minimizing the wait for users when they request information.

More Book Summary





Scan to Download

Listen summary

3.Question

Why is MapReduce described as a significant algorithm for scalability, and what caveats does it carry?

Answer:MapReduce enabled scalable data processing across distributed systems, making it possible to handle massive datasets effectively. However, it is noted to be somewhat simplistic compared to more advanced parallel processing techniques, which can lead to inefficiencies.

4.Question

What is the Unix philosophy mentioned in the text, and how does it influence data processing?

Answer:The Unix philosophy emphasizes building small, modular tools that do one thing well, allowing for easy composition of commands to create powerful data processing pipelines. This philosophy informs both traditional Unix tools and modern frameworks like MapReduce.

5.Question

Can you explain how Stream Processing builds on the concepts of Batch Processing?

Answer:Stream Processing extends Batch Processing by

More Book Summary

Scan to Download

Listen summary

enabling the handling of real-time data, allowing systems to process events shortly after they occur rather than waiting for the accumulation of data in batches. It effectively reduces latency and increases the system's responsiveness.

6.Question

What are some advantages of using batch processing in data systems?

Answer:Batch processing allows for high efficiency in managing large volumes of data, reduces the complexity of processing tasks by avoiding interaction with users during processing, and provides a framework for fault tolerance by maintaining input immutability and output durability.

7.Question

What challenges can arise in batch processing workflows, especially in relation to skewed data?

Answer:Skewed data can lead to imbalanced workloads in batch processing, where some reducers might handle significantly more records than others, slowing down processing times as the whole job may wait for the slowest

More Book Summary
Scan to Download
Listen summary

reducer to finish.

8.Question

How do dataflow engines improve upon traditional MapReduce in terms of task management?

Answer:Dataflow engines streamline processing by managing workflows as a single job, facilitating better optimization, reducing unnecessary materialization of intermediate states, allowing for early task execution once input is ready, and improving resource utilization.

9.Question

Explain how fault tolerance is handled differently between MapReduce and dataflow engines.

Answer:In MapReduce, fault tolerance is achieved through the materialization of intermediate states to HDFS, allowing tasks to restart on different machines without losing data. Dataflow engines, however, often avoid writing intermediate states to HDFS and recompute lost data using a lineage tracking system.

10.Question

What output types are typically produced by batch

More Book Summary
Scan to Download
Listen summary

processing systems?

Answer:Common outputs from batch processing systems include machine learning models, recommendation systems data, search indexes, and compiled databases, which are usually designed for efficient querying and data retrieval in response to user demands.

11.Question

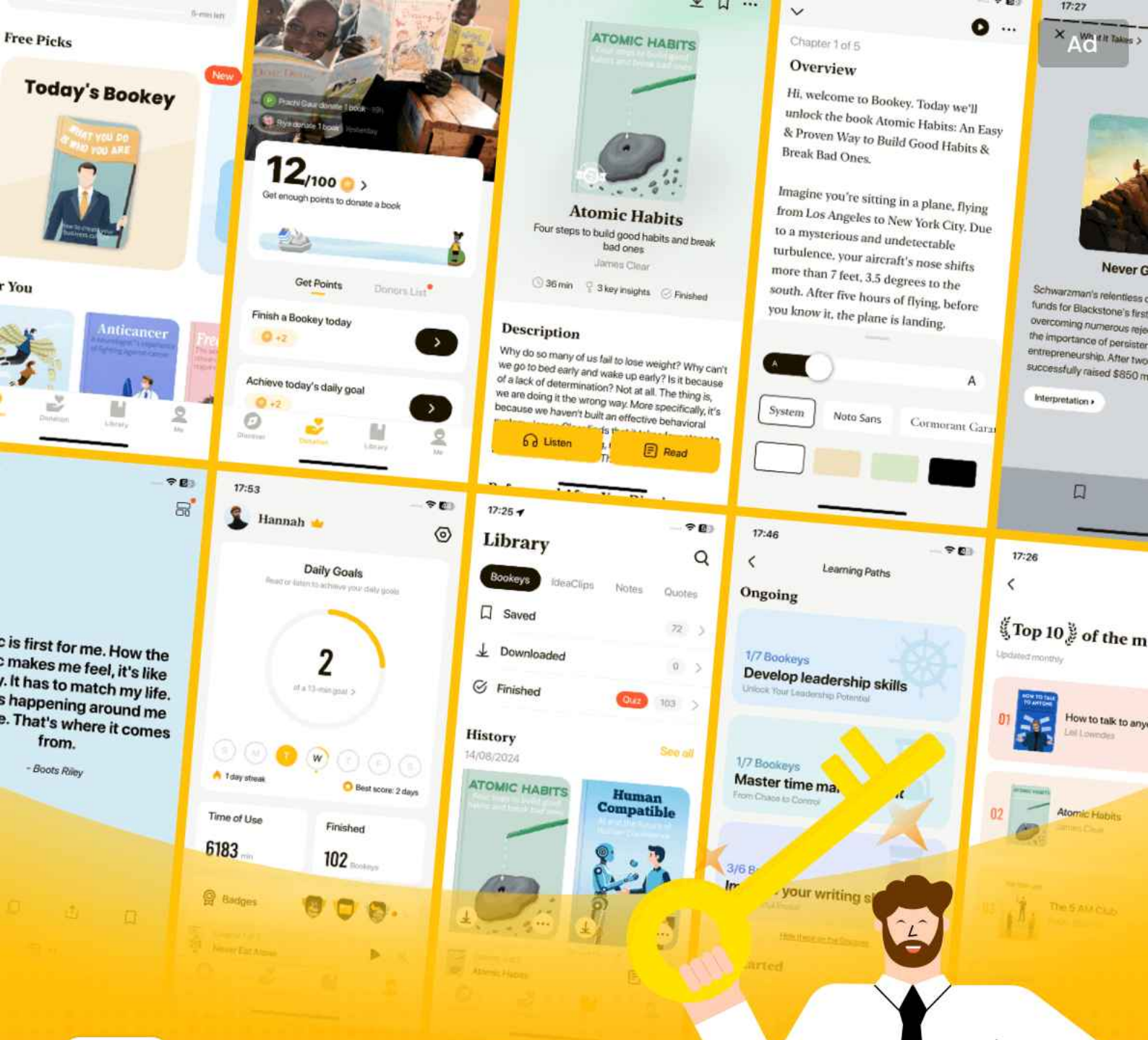
How does the Unix abstraction layer apply to both batch and stream processing?

Answer:Unix's model of chaining simple processes together applies to both batch and stream processing, where the output of one command can be directly used as the input for another, promoting the concept of composability in software design.

More Book Summary

Scan to Download

Listen summary



World' best ideas unlock your potential

Free Trial with Bookey

Download on the App Store

GET IT ON Google Play

Scan to download

Chapter 13 | Stream Processing| Q&A

1.Question

What are the key advantages of stream processing over batch processing?

Answer:Stream processing allows real-time data processing and insights, supporting applications requiring immediate feedback. Unlike batch processing, which operates on fixed datasets that are created and processed over a specific timeframe, stream processing continuously handles unbounded data that flows in over time, enabling more dynamic and timely responses to changing data.

2.Question

How does event timestamping work in stream processing and why is it significant?

Answer:Events in stream processing are typically assigned timestamps based on when they occurred, enabling processors to group and analyze data based on these times. This mechanism is critical for handling time-based queries

More Book Summary

Scan to Download

Listen summary

and analytics accurately, ensuring analytics reflect the actual events rather than when they're processed, which is essential for maintaining data integrity.

3.Question

What challenges arise from dealing with delayed events in stream processing?

Answer:Delayed events, known as stragglers, can lead to inaccuracies in real-time analytics and reporting. Such delays require stream processors to implement mechanisms for handling these late-arriving events, either by adjusting windows or recalculating values, adding complexity to the processing logic.

4.Question

Describe the importance of immutability in the context of event logs and stream processing.

Answer:Immutability ensures that once events are recorded in an event log, they cannot be altered, preserving the history of actions that occurred. This characteristic supports data recovery, auditing, and a reliable process for reconstructing

More Book Summary
Scan to Download
Listen summary

system state, since all historical data is intact and verifiable.

5.Question

What is a 'tumbling window' in stream processing, and how is it applied?

Answer:A tumbling window is a fixed-length time period that groups events into distinct, non-overlapping windows. Each event belongs to exactly one window, making it easier to perform computations like counting or averaging within specific intervals, allowing for efficient batch-like processing of continuous streams.

6.Question

How do stream-table joins differ from traditional batch joins?

Answer:Stream-table joins continuously enrich a stream of events using static data from a database without waiting for a complete set of data. In contrast, traditional batch joins operate on complete datasets at a fixed point in time, inherently less dynamic and less able to adapt as conditions change.

More Book Summary


Scan to Download
Listen summary

7.Question

What are the primary fault tolerance strategies used in stream processing?

Answer:Fault tolerance in stream processing often employs checkpointing, where the state of processing is periodically saved. In case of failure, processing can resume from the last checkpoint, ensuring minimal data loss. Other strategies include microbatching, where streams are processed in small, discrete batches, and the use of idempotent operations ensuring that repeated execution has no side effects.

8.Question

How does the concept of 'log compaction' enhance the efficiency of stream processing?

Answer:Log compaction retains only the most recent state for each key in a log, allowing the system to discard outdated records. This reduces storage requirements and enhances access times by ensuring that only relevant, current data is available for processing, improving overall system efficiency.

More Book Summary

Scan to Download

Listen summary

9.Question

Why is it critical to ensure 'exactly-once' semantics in stream processing?

Answer:'Exactly-once' semantics guarantee that each event is processed precisely once, avoiding duplicates or misses during recovery scenarios. This is crucial for maintaining data integrity, especially in systems that operate on financial transactions or critical data flows where consistency and accuracy are paramount.

10.Question

What role does change data capture (CDC) play in synchronizing databases and streams?

Answer:Change Data Capture (CDC) allows for real-time tracking of changes in a database, producing a stream of change events that can be consumed by other systems. This process helps in maintaining consistency across multiple views or systems by ensuring that they all reflect the most current state of the database.

More Book Summary





Scan to Download

Listen summary

Ad

Scan to Download

Try Bookey App to read 1000+ summary of world best books

Unlock 1000+ Titles, 80+ Topics

New titles added every week

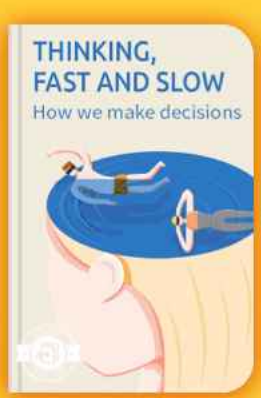
Brand | Leadership & Collaboration | Time Management | Relationship & Communication

ness Strategy | Creativity | Public | Money & Investing | Know Yourself | Positive P

Entrepreneurship | World History | Parent-Child Communication | Self-care | Mind & Spi

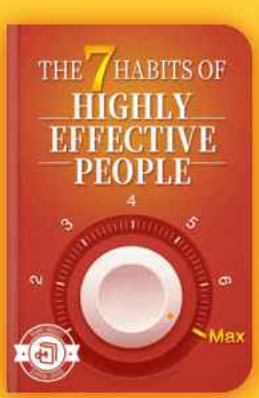
Insights of world best books















Free Trial with Bookey

Designing Data-Intensive Applications Quiz and Test

Check the Correct Answer on Bookey Website

Chapter 1 | Reliable, Scalable and Maintainable Applications| Quiz and Test

1. Applications today are primarily data-intensive rather than compute-intensive.
2. Scalability only concerns the addition of more machines to a system.
3. Good software design enhances maintainability by supporting future modification and adaptation.

Chapter 2 | Data Models and Query Languages| Quiz and Test

1. Layered data models simplify interactions between application developers and database engineers, facilitating better collaboration.
2. The relational model was developed in 1980 by Edgar Codd.
3. Document databases follow a schema-on-write approach,

More Book Summary





Scan to Download

Listen summary

making them less flexible compared to relational databases.

Chapter 3 | Storage and Retrieval| Quiz and Test

1. Different storage engines are optimized for different workloads, particularly transactional (OLTP) and analytical (OLAP) workloads.
2. B-trees are primarily used for their performance in handling raw data storage without any indexing capabilities.
3. Keeping databases in memory can enhance performance but may sacrifice durability during system failures.

More Book Summary
Scan to Download
Listen summary

Download Bookey App to enjoy

1000+ Book Summaries with Quizzes

Free Trial Available!

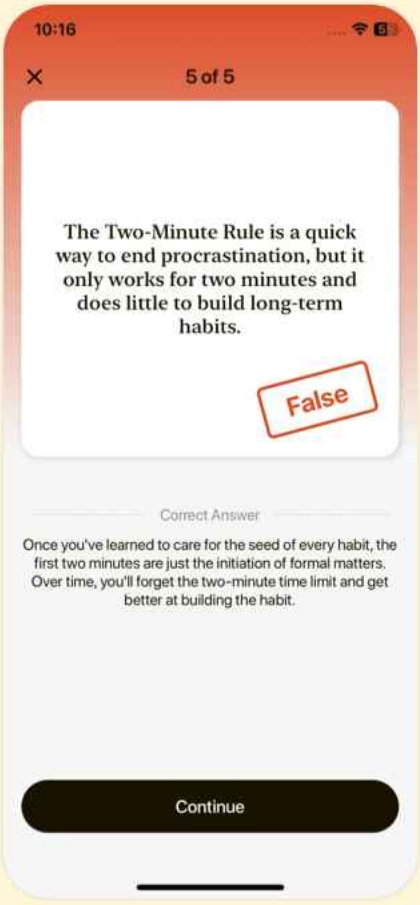
Scan to Download

Download on the App Store

GET IT ON Google Play







Chapter 4 | Encoding and Evolution| Quiz and Test

1. Relational databases operate under a single schema which can be altered, but only one schema is enforced at any given time.
2. Backward compatibility means that older code can read data produced by newer code.
3. Protocol Buffers and Thrift do not rely on schemas for data encoding.

Chapter 5 | Part II. Distributed Data| Quiz and Test

1. Vertical scaling limits the scalability of databases due to costs that can be super-linear.
2. Shared-disk architecture allows for complete independence of each machine’s resources, making it highly scalable without limitations.
3. Data replication in distributed systems is used to enhance redundancy and performance improvements.

Chapter 6 | Replication| Quiz and Test

1. Replication increases availability by allowing the system to continue functioning even if all parts fail.

More Book Summary
Scan to Download
Listen summary

2.In single-leader replication, clients send write requests to the leader, which updates its local storage and informs followers.

3.Synchronous replication guarantees performance and availability at the cost of consistency.

More Book Summary





Scan to Download

Listen summary

Download Bookey App to enjoy

1000+ Book Summaries with Quizzes

Free Trial Available!

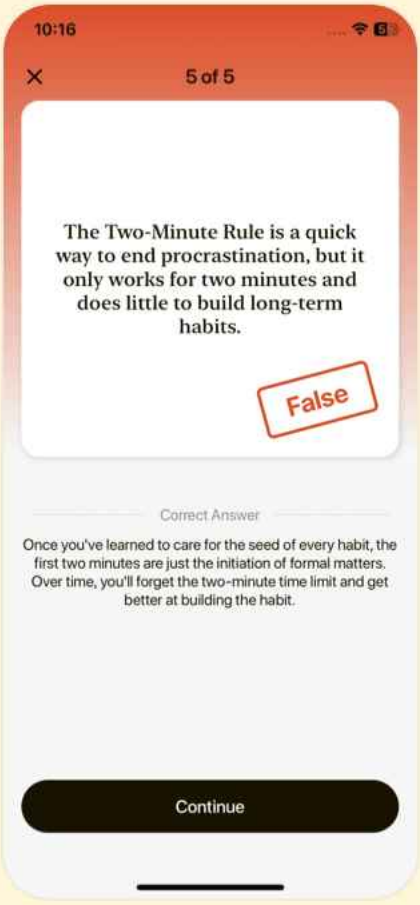
Scan to Download

Download on the App Store

GET IT ON Google Play







Chapter 7 | Partitioning| Quiz and Test

1. The primary goal of partitioning is to achieve fault tolerance in a database.
2. Different databases may use varying terminology for partitioning, such as shards in MongoDB and regions in HBase, but the concept remains consistent.
3. Hash partitioning is most efficient for range queries compared to other partitioning methods.

Chapter 8 | Transactions| Quiz and Test

1. Transactions are designed to ensure data integrity through the ACID properties.
2. Optimistic Concurrency Control prevents all concurrency conflicts by locking resources during transaction processes.
3. Snapshot Isolation allows transactions to view a consistent snapshot of the database, but can lead to write skew when multiple concurrent transactions write conflicting changes.

Chapter 9 | The Trouble with Distributed Systems| Quiz and Test

1. Faults in distributed systems are assumed to be

More Book Summary
Scan to Download
Listen summary

Byzantine according to Chapter 9.

2.Distributed systems can experience partial failures, unlike single computers which are deterministic.

3.Cloud systems and high-performance computing have identical strategies for fault tolerance.

More Book Summary

Scan to Download

Listen summary

Download Bookey App to enjoy

1000+ Book Summaries with Quizzes

Free Trial Available!

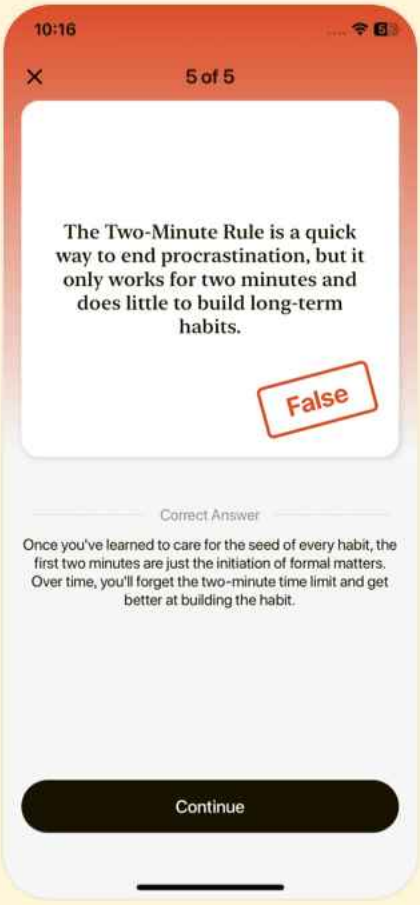
Scan to Download

Download on the App Store

GET IT ON Google Play







Chapter 10 | Consistency and Consensus| Quiz and Test

1. Consensus algorithms are unnecessary in fault-tolerant distributed systems.
2. Linearizability is a strong consistency model that ensures all clients see the same value for a given operation.
3. The two-phase commit (2PC) protocol guarantees that all distributed transactions will always commit successfully, regardless of failures.

Chapter 11 | Part III. Derived Data| Quiz and Test

1. Systems of record are also referred to as the source of truth and maintain the authoritative version of data.
2. Derived data systems always maintain a clear separation from systems of record.
3. Most databases are inherently classified as either systems of record or derived data systems.

Chapter 12 | Batch Processing| Quiz and Test

1. Batch processing systems respond quickly to user

More Book Summary
Scan to Download
Listen summary

requests and are designed for immediate interaction.

2. MapReduce is a batch processing model that has influenced many distributed data processing frameworks.

3. Diverse processing models like Spark and Flink only replicate the functionality of MapReduce without improving its efficiency.

More Book Summary





Scan to Download

Listen summary

Download Bookey App to enjoy

1000+ Book Summaries with Quizzes

Free Trial Available!

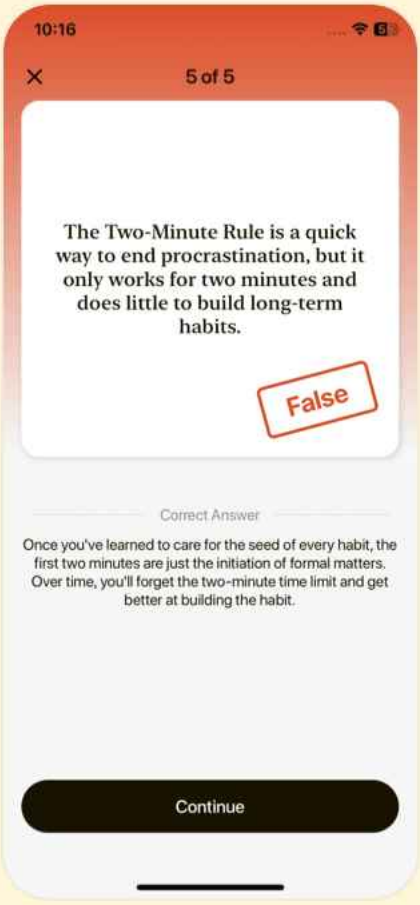
Scan to Download

Download on the App Store

GET IT ON Google Play







Chapter 13 | Stream Processing| Quiz and Test

1. Stream processing deals with finite input data that can be processed all at once.
2. Message brokers help manage message flow and allow producers and consumers to operate independently.
3. Stream processing frameworks must handle the finite nature of data similarly to batch processing frameworks.

More Book Summary
Scan to Download
Listen summary

Download Bookey App to enjoy

1000+ Book Summaries with Quizzes

Free Trial Available!

Scan to Download

Download on the App Store

GET IT ON Google Play





